

Тренажёр оператора ДДС-112

Сопроводительная документация к решению

Хакатон «Лидеры цифровой трансформации 2026»

Задача № 9. Учебное программное обеспечение для подготовки оператора ДДС города Москвы с использованием искусственного интеллекта

Постановщик: Департамент по делам гражданской обороны, чрезвычайным ситуациям и пожарной безопасности города Москвы, ГБУ «Система 112»

Команда «Ультратех»

24.09.2026

Содержание

Содержание.....	2
1. Маршрут проверки решения.....	5
Как открыть	5
Маршрут на 10 минут.....	5
Где ответ по каждому критерию	6
Что стоит заметить.....	6
Если что-то не открылось.....	7
2. Архитектура решения.....	8
Функциональная архитектура.....	8
Компонентная архитектура.....	9
Масштабирование	10
Надёжность	10
Безопасность	10
3. Методы обработки данных	12
1. Данные заказчика.....	12
2. Оценка действий	12
3. Диалог с заявителем.....	14
4. Генерация сценариев	14
5. Аналитика и прогноз	14
4. Установка, настройка и восстановление.....	16
1. Требования.....	16
2. Обычная установка	16
3. Загрузка материалов заказчика.....	16
4. Модели искусственного интеллекта	17
5. Телефония.....	17
6. Установка без доступа в интернет.....	17
7. Резервное копирование и восстановление.....	18
8. Обновление	18
9. Частые проблемы	19
5. Руководство пользователя.....	20
Обучающийся	20

Преподаватель	21
Администратор.....	22
Если что-то не работает.....	23
6. Данные заказчика.....	24
1. Классификатор происшествий (xlsx)	24
2. Билеты (PDF-скан)	27
3. Памятка «Работа на АРМ-112».....	28
4. Улицы Москвы	28
Заложенные ошибки оператора 112 (задача #35)	28
Звонки в службы (задача #36).....	29
7. Условия и ограничения решения.....	30
1. Назначение.....	30
2. Работа без внешних сетей	30
3. Ограничения искусственного интеллекта	30
4. Ограничения телефонии.....	31
5. Ограничения данных	31
6. Что не реализовано	32
7. Условия эксплуатации	32
8. Производительность и нагрузка	33
Задержка ответа.....	33
Точность выбора реплики (select)	33
Провокации (generate).....	35
Как воспроизвести	35
Телефония: задержка ответа в линии (волна 6)	35
Генерация сценариев моделью диалога (волна 11, 20.09.2026)	35
Облачный голос заявителя (Vapi), 21.09.2026.....	36
Вход в систему под нагрузкой (#71, 22.09.2026)	37
Нагрузка по цифрам ТЗ (22.09.2026).....	37
9. Проверка подсчёта оценки на живых звонках	39
Прогон 21.09.2026	39
Пример: стенограмма теста 01	41
Как запустить.....	42

10. Используемые библиотеки и компоненты	43
Бэкенд (Python).....	43
Модели	44
Данные	45
Фронтенд (TypeScript)	45
Инфраструктура (образы).....	45
11. Соответствие техническому заданию	46
1. Ключевые компоненты решения (раздел «Описание работы компании»)	46
2. Предназначение сервиса.....	46
3. Постановка задачи	47
4. Границы решения.....	47
5. Общие требования к сервису	47
6. Опциональные требования.....	48
7. Требования к производительности.....	48
8. Пользовательские требования: интерфейс	49
9. Роль: администратор.....	49
10. Роль: преподаватель.....	50
11. Роль: обучающийся.....	51
12. Нефункциональные требования	52
13. Сценарии из ТЗ и ответы заказчика	53
14. Требования к решению и сдаче	54
Что закрывает оставшиеся пункты.....	54

1. Маршрут проверки решения

Короткий маршрут для эксперта: что открыть, под кем войти, на что смотреть и где лежит ответ по каждому критерию оценки. Полные описания в остальных документах docs/, здесь только навигация.

Что это за система. Учебное программное обеспечение для подготовки диспетчера ДДС города Москвы: имитация рабочего места АРМ-112, звонки, автоматическая оценка действий обучающегося, кабинет преподавателя и прогноз готовности к аттестации. Работает в закрытом контуре без интернета, на процессоре, без видеокарты.

Как открыть

Способ	Что делать
Готовый стенд	Адрес и демонстрационные логины указаны в форме сдачи. Логины и пароли продублированы на странице входа стенда, отдельно запрашивать ничего не нужно
У себя	<code>git clone, cp .env.example .env, docker compose up -d --build</code> , через минуту <code>https://localhost</code> . Подробно в README и INSTALL.md

Роли: **преподаватель** (занятия, отчёты, аналитика), **обучающийся** (рабочее место диспетчера и оператора 112), **администратор** (пользователи, состояние системы, резервные копии).

Маршрут на 10 минут

1. Преподаватель, 2 минуты. Войдите преподавателем.

- «Занятия»: откройте любое завершённое занятие, посмотрите отчёт и выгрузку.
- «Аналитика»: выжимка текстом, тепловая карта «обучающиеся × группы происшествий», динамика по

неделям, типичные ошибки группы, прогноз готовности к аттестации.

- Ссылка «Достоверность прогноза»: качество модели на отложенной выборке, калибровка, вес признаков и честное объяснение, откуда взяты данные.

2. Обучающийся, 5 минут. Войдите обучающимся, откройте активное занятие.

- Режим «Реагирование на карточку»: журнал происшествий, карточка, первичный статус в течение норматива, ход работ, звонок дежурному службы, доклады бригады по телефону.
- Режим «Приём вызова»: звонок заявителя, его роль играет модель, заполнение адреса и опросной карты, выбор служб, сохранение карточки.
- После завершения карточки откройте **разбор**: составляющие оценки, эталон против решения, найденные ошибки со ссылками на страницы памятки заказчика, расшифровка звонков.

3. Администратор, 1 минута. Состояние сервисов, журнал аудита, резервные копии.

4. Код и документы, 2 минуты. Разделы ниже.

Где ответ по каждому критерию

Критерий экспертизы	Где смотреть
Идея и способ решения, используемые технологии	ARCHITECTURE.md, LIBRARIES.md
Качество кода	backend/app и frontend/src, тесты: 64 набора на сервере и 19 в интерфейсе; запуск в README
Возможность интеграции в другие системы	HTTP API с описанием OpenAPI на /api/openapi.json (страница /api/docs включается вне боевого режима), обмен по протоколам SIP и ARI, адрес модели задаётся настройкой, см. ARCHITECTURE.md
Скорость работы решения	PERFORMANCE.md: отклик при 100 пользователях, 20 одновременных занятий, запись в базу, задержка голоса, время ответа модели на процессоре
Проверка работы расчётов	VOICE_TESTS.md: 12 живых звонков, 408 проверок подсчёта баллов без расхождений с независимым пересчётом; формулы в METHODS.md, раздел 2
Проверка прогнозирования и достоверности	Страница «Достоверность прогноза» в кабинете преподавателя; методика в METHODS.md, раздел 5
Объективность диаграмм	Раздел «Аналитика»: под каждым графиком написано, из каких данных он построен и за какой период
Соответствие техническому заданию	REQUIREMENTS_MATRIX.md: каждый пункт ТЗ, где реализован и как проверить
Полнота сопроводительной документации	docs/: архитектура, методы, установка, руководство пользователя, ограничения, данные, библиотеки
Честные границы решения	LIMITATIONS.md: что система не делает и почему
Работа с данными заказчика	DATASET.md: классификатор происшествий, памятка, учебные билеты, что загружено и как

Что стоит заметить

- **Интерфейс повторяет живой АРМ-112.** Реестр происшествий и карточка сделаны по скриншотам

заказчика от 17.09.2026: те же колонки, названия статусов и расположение блоков. Преподаватель из ГБУ «Система 112» узнаёт своё рабочее место без обучения.

- **Работает без интернета и без видеокарты.** По умолчанию распознавание речи, языковая модель,

синтез голоса и проверка грамотности подняты внутри контура. Адрес модели задаётся настройкой, поэтому её можно вынести на отдельный сервер, в том числе с видеокартой.

- **Телефония настоящая.** Звонки идут через SIP: программный телефон в браузере, настольный IP-телефон, звонок диспетчера дежурному службы и обратный звонок заявителю.

- **Оценка объяснимая.** Балл раскладывается на составляющие, каждая ошибка названа словами памятки заказчика со ссылкой на страницу, преподаватель может изменить итог с указанием причины.

Если что-то не открылось

Запасной путь: видеозапись демонстрации, ссылка в поле «Дополнительные материалы» формы сдачи. Установка с нуля описана в INSTALL.md, включая перенос без интернета и проверку восстановления из резервной копии.

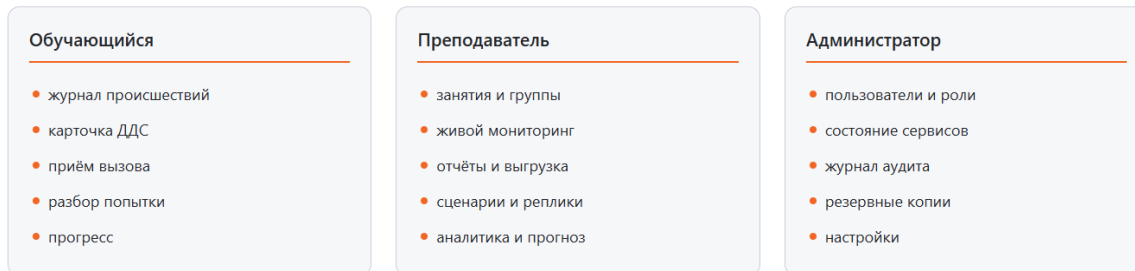
2. Архитектура решения

Тренажёр оператора ДДС-112 работает в закрытом контуре: все компоненты, включая модели искусственного интеллекта и телефонию, находятся внутри локальной сети учебного центра и не обращаются во внешние сети.

Функциональная архитектура

Система обслуживает три роли и два режима занятий.

Функциональная архитектура: роли и разделы



Роли и разделы системы

Обучающемуся доступны журнал происшествий, карточка ДДС, приём вызова, разбор попытки и собственный прогресс. Преподавателю доступны занятия и группы, живой мониторинг, отчёты и выгрузка, сценарии и реплики, аналитика с прогнозом. Администратору доступны пользователи и роли, состояние сервисов, журнал аудита, резервные копии и настройки.

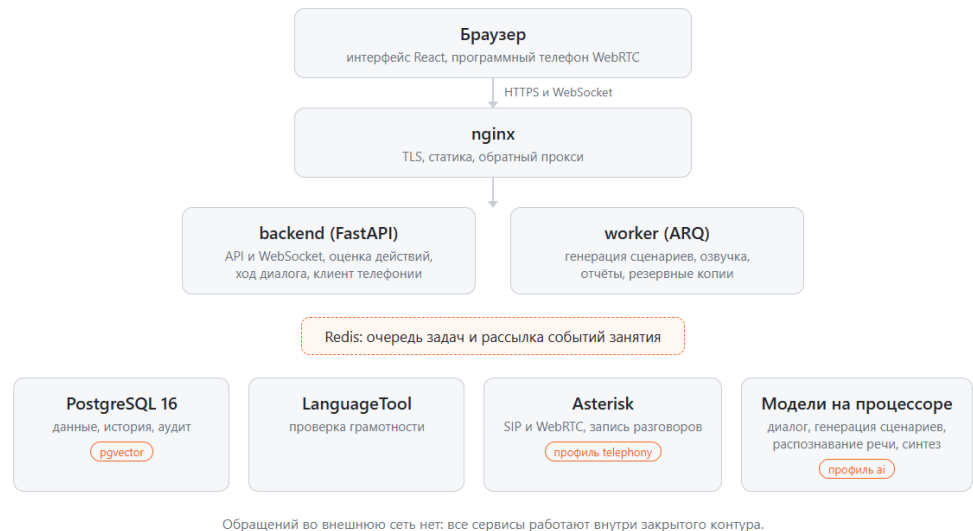
Режим «Реагирование на карточку». Карточка происшествия приходит в журнал службы обучающегося. Он открывает её, проверяет данные, принимает или не принимает в течение норматива, ведёт ход работ статусами с комментариями и номером наряда.

Режим «Приём вызова». Обучающийся принимает звонок, роль заявителя исполняет система, и заполняет карточку: адрес, опросную карту признаков, флаги, описание. Службы для оповещения подставляются автоматически по классификатору.

После завершения попытки движок оценки сравнивает действия с эталоном сценария и выдаёт разбор.

Компонентная архитектура

Компонентная архитектура



Компонентная архитектура

Браузер работает с сервером по HTTPS и WebSocket через nginx, который завершает TLS и отдаёт статику. Запросы обслуживает backend на FastAPI: API, события занятия, оценка действий, ход диалога и клиент телефонии. Фоновые задачи (генерация сценариев, озвучка, отчёты, резервные копии) выполняет worker, очередь и рассылка событий идут через Redis. Данные, история и аудит лежат в PostgreSQL с расширением pgvector, грамотность проверяет LanguageTool, телефонию ведёт Asterisk (профиль telephony), модели диалога, генерации, распознавания и синтеза речи работают на процессоре (профиль ai). Обращений во внешнюю сеть нет.

Сервисы Docker Compose

Сервис	Профиль	Назначение
nginx	всегда	TLS, статика интерфейса, прокси к /api и /ws
backend	всегда	REST API, WebSocket, оценка, диалог, клиент телефонии
worker	всегда	фоновые задачи: генерация, озвучка, отчёты, копии
postgres	всегда	база данных, расширение pgvector для поиска по методичкам
redis	всегда	очередь задач и рассылка событий занятия
languagetool	всегда	проверка грамотности текстов обучающегося
backup	всегда	ежесуточные резервные копии базы и файлов
llm-dialog	ai	модель, выбирающая реплику заявителя
llm-gen	ai	модель для генерации сценариев
stt	ai	распознавание речи обучающегося
asterisk	telephony	SIP-сервер, WebRTC, запись разговоров, интерфейс ARI

Профили live и monitoring зарезервированы под узел с видеокартой и под связку Prometheus с Grafana.

Данные

Что	Где
Пользователи, группы, занятия, попытки, оценки	PostgreSQL
Справочники заказчика: классификатор, службы, статусы, типичные ошибки	PostgreSQL, загружаются импортёром из файлов заказчика
Сценарии и их версии	PostgreSQL, тело сценария в JSONB
Методички для поиска и подсказок	PostgreSQL, векторные представления в pgvector
Записи разговоров, озвученные реплики, отчёты	том <code>storage</code>
Резервные копии	том <code>backups</code>
Журнал аудита	PostgreSQL, отдельная таблица без права на изменение и удаление

Масштабирование

Вертикальное. Больше ядер и памяти ускоряют модели и позволяют увеличить число одновременных занятий.

Горизонтальное. Бэкенд не хранит состояния между запросами, поэтому его можно запускать в нескольких экземплярах за nginx. Телефония и модели выносятся на отдельные узлы той же сети: адреса задаются переменными окружения, код при этом не меняется. Типовая раскладка стенда:

- узел приложения: nginx, backend, worker, postgres, redis, languagetool, backup;
- узел телефонии: asterisk (открыты только SIP и диапазон медиапортов);
- узел моделей: llm-dialog, llm-gen, stt.

Надёжность

- У каждого сервиса есть проверка работоспособности, при сбое контейнер перезапускается.
- Резервное копирование базы и файлов выполняется по расписанию, копии хранятся заданное число

дней, восстановление проверяется скриптом `scripts/restore_test.sh`.

- Обрыв связи до 30 секунд не приводит к потере данных: черновик карточки хранится в браузере и

на сервере, отправка идемпотентна, а события занятия догоняются по порядковому номеру.

- Журнал аудита связан цепочкой контрольных сумм, нарушение цепочки обнаруживается проверкой.

Безопасность

- Доступ по ролям с проверкой владельца объекта на стороне сервера.
- Пароли хранятся в виде хешей, вход ограничен по числу попыток.
- Трафик внутри контура защищён TLS, голос в браузере идёт по защищённому WebRTC.
- Персональные данные минимальны: показ ФИО отдельным действием, которое пишется в журнал аудита.
- Внешние обращения запрещены на уровне конфигурации: при попытке указать внешний адрес модели

без явного разрешения система не запускается.

3. Методы обработки данных

Документ описывает, как система обрабатывает данные заказчика, как устроена оценка действий обучающегося, как работает диалог с заявителем и как строится прогноз готовности.

1. Данные заказчика

Классификатор происшествий. Таблица заказчика разбирается программно: три уровня признаков («что», «где», уточнение) образуют дерево опросной карты, итоговый тип происшествия и набор служб берутся из тех же строк. У части служб есть отдельные колонки под дополнительные признаки («Пострадавшие», «Нет доступа», «Угроза людям», «Правонарушение»), поэтому список оповещаемых служб зависит не только от типа, но и от выбранных флагов.

Правило выбора служб хранится в виде списка условий: первое подходящее выигрывает, последнее является значением по умолчанию. Функция подбора служб покрыта табличным тестом на строках из исходного файла.

Памятка «Работа на АРМ-112». Из неё взяты статусы реагирования, правила их простановки, статусы карточки и перечень типичных нарушений с примерами. Тексты используются и в оценке, и в справочнике, доступном обучающемуся.

Билеты. Учебные ситуации заказчика приходят сканом, поэтому страницы поворачиваются и распознаются, результат сверяется и хранится с отметкой уверенности распознавания. Каждая ситуация превращается в сценарий: лист фактов для заявителя, эталонная карточка и эталонные действия.

2. Оценка действий

Оценка построена как набор независимых составляющих. Каждая даёт баллы и объяснение, итог приводится к ста баллам, зачёт определяется порогом занятия.

Режим «Реагирование на карточку»:

Составляющая	Что проверяет
Решение	принял или не принял карточку, совпадает ли причина отказа с эталонной
Время принятия	сколько прошло от поступления карточки до первичного статуса
Цепочка статусов	поставлены ли обязательные статусы и в правильном ли порядке
Комментарии	есть ли обязательные комментарии, насколько они близки по смыслу к эталону
Типичные ошибки	срабатывание правил, описанных в памятке
Грамотность	орфография и пунктуация в текстах обучающегося

Режим «Приём вызова»: путь признаков в опросной карте, флаги и список служб, адрес, обязательные вопросы заявителю, описание, время, типичные ошибки, грамотность.

Как считаются отдельные вещи.

- ***Время.*** Полный балл, если уложился в норматив; ноль, если превысил вдвое; между ними линейно.

- *Текст.* Сравнение ответа с эталоном по смыслу: тексты переводятся в числовые представления,

считается косинусная близость. Пороги подобраны так, чтобы пересказ своими словами засчитывался, а пустая отписка нет. Если модель представлений недоступна, используется частотный метод, и в результате указывается, каким способом считали.

- *Адрес и названия.* Приводятся к общему виду: регистр, буква «ё», типы улиц, сокращения; затем

сравниваются с допуском на опечатку.

- *Типичные ошибки.* Каждое правило из памятки это отдельная функция: например, первичный статус

позже норматива, отказ без комментария, пропущенные статусы хода работ, отказ от профильного происхождения. Каждое правило покрыто тестом.

- *Недоступный сервис.* Если проверка грамотности недоступна, её составляющая помечается «не

проверено», а итог пересчитывается по оставшимся, чтобы обучающийся не терял баллы из-за инфраструктуры.

- *Блокирующие правила.* Две ошибки в приёме вызова критические: при них итог «незачёт» при

любом балле. **Обязательные вопросы не заданы** — обучающийся сам задал меньше половины обязательных вопросов (доля задаётся в сценарии полем `min_questions_share`, по умолчанию 0,5; в облачном звонке темы, которые заявитель сообщил сам без вопроса, в зачёт не идут, потому что темы там определяются по ключевым словам в свободной речи). **Карточка не заполнена** — нет ни типа происхождения, ни адреса, ни описания. В разборе причина видна текстом: «Незачёт: задано 0 из 7 обязательных вопросов (нужно не меньше 4)».

- *Пустой ввод.* Составляющая, которой нечего оценивать, получает 0, а не максимум: описание короче двух слов даёт 0 по описанию и по грамотности («нет ошибок» не значит «нет текста»); незаполненная карточка даёт 0 ещё по признакам и службам и по времени (быстро сохранить пустую карточку — не заслуга). Обязательные вопросы при этом считаются по разговору как обычно, так что пустая карточка после полноценного разговора даёт около 15 баллов и незачёт.

Оценка не зависит от базы данных и внешних вызовов: это чистые функции, поэтому её поведение воспроизводимо и проверяется тестами. Сквозная проверка на живых звонках с облачным заявителем — двенадцать сценариев, 408 сверок итога и составляющих с независимым пересчётом, без расхождений — описана в `VOICE_TESTS.md`.

3. Диалог с заявителем

Заявителя играет система. Сценарий содержит лист фактов (что случилось, адрес, пострадавшие, имя, поведение) и набор реплик, утверждённых преподавателем.

Режимы работы:

- **выбор реплики** (основной): модель получает вопрос обучающегося и пронумерованный список

утверждённых реплик и возвращает номер. Текст не генерируется, поэтому заявитель физически не может сообщить то, чего преподаватель не утверждал;

- **свободная генерация**: модель отвечает своими словами, опираясь только на лист фактов;
- **смешанный**: сначала выбор, при отсутствии подходящей реплики генерация, новая реплика

отправляется преподавателю на утверждение;

- **без модели**: вопросы задаются кнопками, ответы берутся из листа фактов. Режим нужен для запуска на слабой машине и не используется на занятиях.

Темы вопросов определяются двумя способами сразу: ответом модели и поиском ключевых слов. Это защищает оценку обязательных вопросов от ошибок модели.

Речь обучающегося распознаётся локально, ответ заявителя озвучивается заранее при утверждении сценария, поэтому во время разговора синтез не выполняется и задержка остаётся небольшой.

4. Генерация сценариев

Преподаватель описывает ситуацию фразой или берёт билет заказчика. Система подбирает подходящие фрагменты памятки и похожие билеты, модель формирует сценарий по строгой схеме: лист фактов, реплики, эталонная карточка, эталонные действия. Ответ проверяется схемой и справочниками: коды типов, служб и статусов должны существовать, выдуманные значения отклоняются. При ошибке разбор повторяется с указанием, что именно неверно.

Преподаватель правит, проверяет грамотность и утверждает сценарий целиком или по частям. Утверждённые части при последующих переделках не меняются.

5. Аналитика и прогноз

Рейтинг навыка считается по каждой группе происшествий и режиму: результат сравнивается с ожидаемым для текущего уровня, рейтинг растёт сильнее за успех на сложном сценарии.

Адаптивный подбор выдаёт следующую карточку из той группы, где рейтинг ниже, и по сложности, соответствующей уровню.

Прогноз готовности к аттестации строится на признаках: средний балл последних занятий, направление изменения, доля времени от норматива, неверные решения, пропущенные обязательные вопросы, частота типичных ошибок, число попыток. Модель простая и объяснимая,

поэтому для каждого прогноза можно назвать два признака, которые сильнее всего тянут результат вниз.

Проверка достоверности. Модель учится на ранних занятиях обучающегося и предсказывает результат следующего, которого не видела. Сравниваются прогноз и факт, считаются средняя ошибка в баллах и попадание в интервал, результат сравнивается с простыми способами предсказания («средний балл», «последний балл»). Все числа публикуются как есть на странице «Достоверность прогноза».

4. Установка, настройка и восстановление

Документ описывает установку в закрытом контуре, перенос без интернета, настройку телефонии, резервное копирование и восстановление.

1. Требования

Сервер приложения (минимум по техническому заданию):

- процессор от 6 ядер (Intel Core i7, Xeon или аналог);
- оперативная память от 32 ГБ;
- накопитель NVMe SSD от 512 ГБ;
- операционная система Ubuntu 20.04 и новее либо Windows 10/11;
- Docker и Docker Compose.

Без моделей искусственного интеллекта (профиль ai выключен) система работает и на машине с 8 ГБ памяти: заявитель в этом режиме отвечает заранее утверждёнными репликами.

Рабочее место обучающегося: браузер Chrome, Firefox или Яндекс.Браузер, гарнитура с микрофоном. Настольный IP-телефон подключается дополнительно.

2. Обычная установка

```
git clone <адрес репозитория> lct-112-simulator
cd lct-112-simulator
cp .env.example .env
```

В файле .env обязательно поменять SECRET_KEY, POSTGRES_PASSWORD, APP_DB_PASSWORD, SEED_PASSWORD, ARI_PASSWORD. Для боевой установки поставить DEMO_MODE=false: тогда при первом запуске создаётся только администратор, а демонстрационные пользователи и история не создаются.

```
docker compose up -d --build
curl -sk https://localhost/health
```

Интерфейс открывается по адресу https://localhost. Сертификат выпускает внутренний центр сертификации; чтобы браузер не предупреждал, установите корневой сертификат:

```
docker compose cp nginx:/etc/nginx/certs/ca.crt .
```

3. Загрузка материалов заказчика

Положите файлы заказчика в каталог data/organizers/ (классификатор происшествий, памятку, билеты) и выполните:

```
docker compose exec backend python -m app.importers.organizers
docker compose exec backend python -m app.seed
```

Импортер идемпотентен: повторный запуск не создаёт дубликатов. Итог разбора записывается в docs/DATASET.md.

4. Модели искусственного интеллекта

```
./scripts/fetch_models.sh          # требуется интернет, около 10 ГБ
docker compose --profile ai up -d
```

Скрипт скачивает модель для выбора реплик заявителя, модель для генерации сценариев, модель распознавания речи и голоса синтеза. Перечень и контрольные суммы лежат в `scripts/models.manifest`, лицензии перечислены в `docs/LIBRARIES.md`.

Настройки в `.env`: `LLM_DIALOG_MODEL`, `LLM_GEN_MODEL`, `STT_MODEL`, число потоков `LLM_THREADS`, режим диалога `DIALOG_MODE`, движок голоса заявителя `TTS_PROVIDER` (auto: Silero при наличии `tts/silero_v4_ru.pt`, иначе Piper, иначе только текст). Голос Silero скачивается с `models.silero.ai` — если этот адрес закрыт, скачайте файл на другой машине и положите в `models/tts/`.

5. Телефония

```
docker compose --profile telephony up -d
```

В `.env` задать `TELEPHONY_ENABLED=true`, внешний адрес сервера `TELEPHONY_EXTERNAL_IP`, диапазон медиапортов `TELEPHONY_RTP_START` и `TELEPHONY_RTP_END`, порт SIP `SIP_PORT`, пароль интерфейса управления `ARI_PASSWORD`.

На межсетевом экране открыть: порт SIP (по умолчанию 5060, UDP и TCP) и диапазон медиапортов (UDP).

Софтфон в браузере настраивается автоматически: учётная запись обучающегося создаётся вместе с пользователем, данные приходят при входе.

Настольный IP-телефон (например, РТУ Т16Р): в настройках аппарата указать адрес сервера (адрес узла телефонии), имя пользователя и пароль из карточки обучающегося в кабинете администратора, кодеки G.722 и PCMA, регистрация по UDP.

Если телефония выключена (`TELEPHONY_ENABLED=false`), занятия работают полностью: вызов имитируется в интерфейсе, речь передаётся через микрофон браузера.

6. Установка без доступа в интернет

Сборка образов требует интернета (пакеты Python и prtm, базовые образы), поэтому поставка собирается на машине с интернетом и переносится в контур готовой. В контуре ничего не скачивается и не собирается.

На машине с интернетом (Docker, git, около 12 ГБ на диске с моделями):

```
./scripts/fetch_models.sh          # один раз, ~10 ГБ
./scripts/offline_bundle.sh --with-models  # ./offline-bundle: images.tar, source.tar, models.tar,
                                           # organizers.tar (если есть data/organizers),
install.sh,                        # SHA256SUMS, VERSION
```

Без `--with-models` поставка занимает около 2 ГБ и работает без профиля `ai` (заявитель отвечает утверждёнными репликами). Образы всех профилей (`ai`, `telephony`) входят всегда.

На сервере в контуре (Docker и docker compose v2, больше ничего):

```
./install.sh --dir /opt/lct/app # минимальная установка
./install.sh --dir /opt/lct/app --profile ai --profile telephony
```

Установщик сверяет контрольные суммы, загружает образы (`docker load`), распаковывает исходники, модели и материалы заказчика, создаёт `.env` из `.env.example`, если его нет, и поднимает систему с `--pull never --no-build`: любая попытка что-то скачать завершилась бы ошибкой, а не тихим обращением в сеть. При первом запуске бэкенд применяет миграции, загружает классификатор и создаёт пользователей. Дальше — раздел 2: поменять `SECRET_KEY` и пароли в `.env`, `DEMO_MODE=false` для боевой установки, перезапустить.

Образы собранных сервисов имеют постоянные имена (`lct112/backend`, `lct112/nginx`, `lct112/backup`, `lct112/stt`, `lct112/asterisk`; тег `APP_VERSION`, по умолчанию `latest`), поэтому установка не зависит от имени каталога или проекта `compose`.

Проверено 22.09.2026: поставка без моделей (1,8 ГБ образов) развёрнута на стенде в чистый каталог с удалёнными образами и `--pull never`; журнал Docker за время установки не содержит ни одного обращения к реестру; `/health ok`, страница входа 200, e2e-сценарий «полный проход» ДДС проходит.

7. Резервное копирование и восстановление

Копии создаются ежедневно сервисом `backup` в том `backups`; время задаётся `BACKUP_TIME`, число хранимых копий `BACKUP_COUNT`. Копию можно создать вручную в кабинете администратора.

Проверка восстановления:

```
./scripts/restore_test.sh
```

Скрипт разворачивает последнюю копию в отдельную базу и проверяет, что данные читаются. Полное восстановление: остановить приложение, восстановить базу из копии, распаковать файлы тома `storage`, запустить приложение.

8. Обновление

```
git pull
docker compose up -d --build
docker compose exec backend alembic upgrade head
```

Миграции базы выполняются автоматически при старте бэкенда; команда нужна только при ручном обновлении.

Если обновление сменило голосовой движок (например, появился `tts/silero_v4_ru.pt`), уже озвученные реплики остаются в старом голосе — перезвучить их текущим движком:

```
./scripts/fetch_models.sh --only tts
docker compose exec worker python -m app.scripts.revoice --dry-run # сколько реплик
docker compose exec worker python -m app.scripts.revoice
```

Записи, загруженные преподавателем вручную, команда не трогает.

9. Частые проблемы

Симптом	Причина и решение
Браузер предупреждает о сертификате	Установить корневой сертификат из тома <code>certs</code>
Звонок не приходит в браузер	Проверить <code>TELEPHONY_ENABLED</code> , профиль <code>telephony</code> , доступность порта SIP и диапазона медиапортов
Заявитель отвечает не по теме или молчит	Проверить, что поднят профиль <code>ai</code> и модели скачаны; при недоступной модели система переходит на утверждённые реплики и сообщает об этом
Оценка не проверяет грамотность	Недоступен сервис <code>languageTool</code> : составляющая помечается «не проверено», остальная оценка работает
Долгий первый ответ после запуска	Модели прогреваются; повторные ответы быстрее
Не хватает памяти на ноутбуке	Запускать без профиля <code>ai</code>

5. Руководство пользователя

Как пользоваться тренажёром. Разделы по ролям: обучающийся, преподаватель, администратор.

Вход: откройте адрес системы, введите логин и пароль, выданные администратором. При первом входе система попросит сменить пароль.

Обучающийся

Мои задания

После входа видно активное занятие и список назначенных. Кнопка «Перейти к рабочему месту» открывает эмулятор.

Рабочее место ДДС: реагирование на карточку

1. В журнале появляются карточки происшествий вашей службы. У каждой идёт таймер принятия.
2. Откройте карточку: слева заявитель, адрес и описание, справа признаки происшествия и флаги, внизу полоса служб.
3. Во вкладке своей службы нажмите карандаш и поставьте **«Принята»** или **«Не принята»**.

При отказе обязательно выберите причину и напишите комментарий: что именно не в вашей компетенции и кому передана информация.

4. Дальше ведите ход работ по одному шагу, как на АРМ-112: «Начало реагирования» (здесь указывается номер наряда), «Прибытие»,

«Проведение работ», «Работы завершены». Комментарий поясняет, что сделано.

5. Если работы не проводились, поставьте «Отказ от выполнения работ» с комментарием.

Горячие клавиши: Alt+A принять, Alt+R не принять, Ctrl+Enter сохранить, ? подсказка.

Важно: первичный статус нужно поставить в течение норматива, по умолчанию 30 секунд. Если не успеть, карточка получает отметку «Не оповещено», и это учитывается в оценке.

Рабочее место оператора: приём вызова

6. Поступает вызов, нажмите «Ответить».
7. Разговаривайте с заявителем: говорите в гарнитуру или пишите в поле разговора. Спрашивайте по

существу: что случилось, точный адрес, есть ли пострадавшие, есть ли угроза, как зовут заявителя.

8. Заполняйте карточку по ходу разговора: фамилия и статус заявителя (очевидец, пострадавший, родственник, знакомый, ребёнок, участник), адрес (округ и район подставятся по улице), описание со слов заявителя.

9. Тип происшествия: наберите часть слова в строке «что случилось?» — «газ оборуд» найдёт «Запах

от газового оборудования», Enter выберет первый вариант, опросная карта заполнится сама; либо «по группам» и дальше по кнопкам. Пока тип не выбран, под картой написано, что уточнить.

10. «Пострадавшие» спросит, сколько их. Службы подставятся автоматически по признакам; «+» в полосе служб открывает окно «Добавьте службы» с поиском — автоматические подсвечены синим и не снимаются, остальные добавляются и убираются щелчком, «Сохранить и закрыть».

11. Если заявитель бросил трубку или связи нет, отметьте это кнопками «срыв звонка» или «нет контакта».

12. Нажмите «Сохранить» и подтвердите «Оповестить и сохранить карточку» (Ctrl+Enter дважды).

Разбор

Сразу после завершения открывается разбор: итоговый балл, из чего он сложился, что сделано верно, какие ошибки сработали и почему, время против норматива, стенограмма разговора и запись. Читайте объяснения: они написаны словами памятки, по которой работают настоящие диспетчеры.

Мой прогресс

Динамика баллов, время, сильные и слабые группы происшествий, частые ошибки и рекомендации.

Справочник

Памятка «Работа на АРМ-112» целиком, постранично, и методические материалы, которые загрузил преподаватель, — тоже целиком. Ниже поиск по словам сразу по памятке, материалам и классификатору происшествий; из результата открывается сам документ.

Преподаватель

Занятие

13. «Занятия» → «Создать».

14. Выберите режим (реагирование на карточку или приём вызова), группу, категории происшествий,

сложность, профиль службы, норматив и порог зачёта. Подсказки можно выключить для аттестации.

15. «Начать занятие»: обучающиеся получают карточки или вызовы.

16. Во время занятия открыт живой мониторинг: у каждого обучающегося видно состояние, таймер и

последний балл. Можно открыть его попытку и смотреть в реальном времени.

17. «Завершить занятие», затем «Отчёт».

Отчёт

По каждому обучающемуся: число попыток, средний балл, среднее время, отклонение от норматива, ошибки и грамотность. Строка раскрывается до отдельных попыток; у попытки колонка «Действия» раскрывает, что обучающийся делал и когда: статусы с нарядом и комментарием, отметки ошибок в карточке, звонки в службы, в приёме вызова — вопросы заявителю. Отчёт выгружается в PDF, Excel и CSV, действия попадают в выгрузку.

Оценку можно изменить, указав причину: исходная оценка сохраняется, изменение фиксируется в журнале аудита и видно обучающемуся.

Сценарии

- «Сгенерировать»: опишите ситуацию фразой, выберите тип и сложность. Система подготовит сценарий:

лист фактов заявителя, его реплики и эталон.

- Прослушайте реплики, поправьте текст, при необходимости загрузите свою запись голосом.
- «Что исправить» плюс «Переделать»: система учтёт замечание и сделает новую версию, прежние

сохранятся.

- «Проверить грамотность» подсветит ошибки в текстах.
- «Утвердить» целиком или по частям. Только утверждённые сценарии попадают в занятия.
- «Удалить»: сценарий, по которому никто не занимался, удаляется совсем; сценарий из прошлых

занятий уходит в архив (фильтр «Статус: В архиве»), чтобы разборы этих занятий остались открытыми. Из архива сценарий можно «Восстановить».

- Сюда же можно загрузить методические материалы (PDF, DOCX, TXT): обучающиеся читают их в

«Справочнике» целиком и находят поиском, а генерация сценариев учитывает их как контекст.

Аналитика

Тепловая карта группы по категориям происшествий, динамика, типичные ошибки с пояснением, прогноз готовности к аттестации по каждому обучающемуся с объяснением, почему прогноз такой. Отдельная страница «Достоверность прогноза» показывает, насколько прогнозы совпадали с фактом.

Администратор

- **Пользователи:** создание, роль, служба, блокировка, сброс пароля. Фамилия и имя скрыты, показ

фиксируется в журнале аудита.

- **Состояние системы:** сервисы, нагрузка, активные занятия и звонки, оповещения о сбоях.
- **Журнал аудита:** фильтры по пользователю, действию и дате; кнопка «Проверить целостность»

подтверждает, что записи не изменялись.

- **Резервные копии:** список, создание копии вручную, расписание.
- **Настройки:** телефония, журналирование, параметры копирования.

Администратор намеренно не может менять оценки и сценарии: это зона преподавателя.

Если что-то не работает

Что видно	Что делать
Не приходит вызов	Проверьте статус телефонии в шапке; он должен быть «доступен»
Заявителя не слышно	Проверьте выбор устройства и разрешение браузера на микрофон
Заявитель отвечает не в попад	Переформулируйте вопрос короче; сообщите преподавателю, он добавит реплику
Карточка не сохраняется	Проверьте обязательные поля, они подсвечены
«Карточка уже закрыта»	Занятие завершено преподавателем или карточка отправлена ранее

6. Данные заказчика

Файлы организаторов лежат в `data/organizers/` (в `git` не попадают). Всё, что из них получено программно, лежит в `data/seed/` и коммитится: стенд поднимается без файлов организаторов. Импорт в базу: `docker compose exec backend python -m app.importers.organizers` (идемпотентно; если `xlsx` на месте, `seed/classifier.json` пересобирается).

Файл организаторов	Что внутри	Куда ложится
Классификатор_происшествий_v046.xlsx	1310 строк × 99 колонок, лист «Лист1»	<code>incident_groups</code> , <code>incident_types</code> , <code>incident_flags</code> , <code>services</code> ; <code>seed/classifier.json</code>
Билеты-задачи_по_C112_АГС_ГСИ.pdf	32 страницы-скана, по 3 ситуации	<code>tickets</code> ; <code>seed/tickets.json</code> (+ <code>seed/tickets_corrections.json</code>)
Работа_с_АРМ-112_для_ДДС.pdf	памятка, 40 страниц с текстовым слоем	<code>response_statuses</code> , <code>card_statuses</code> , <code>reject_reasons</code> , <code>typical_errors</code> ; <code>seed/memo.txt</code> ; скриншоты в <code>docs/reference/arm112/</code>
— (OpenStreetMap)	улицы Москвы	<code>streets</code> ; <code>seed/streets.json</code>

1. Классификатор происшествий (xlsx)

Структура

Лист «Лист1», 99 колонок (в PRD было сказано 104, фактически 99: А–СU). Строки 1–3 — шапка с объединёнными ячейками, данные с 4-й строки, всего 1307 строк данных.

Колонки	Содержимое	Куда
A–D	коды Г, п1, п2, п3	код типа Г.п1.п2.п3, например 1.1.1.1
E	формула номера ($G \cdot 10^6 + p1 \cdot 10^4 + p2 \cdot 10^2 + p3$); у строк-заголовков групп — порядковый номер группы	не используется (код собирается из A–D)
F	у строки-заголовка группы — название группы; у строк типов — подгруппа «учёт в статистике», заполнена на первой строке подгруппы	<code>incident_groups.title</code> ; <code>incident_types.stat_group</code> (протягивается вниз)
G, H, I	112-Признак.1 / .2 / .3 — кнопки опросной карты	<code>sign1</code> , <code>sign2</code> , <code>sign3</code>
J	дополнительные признаки, не влияющие на тип (подсказки оператору: «В: кто сообщает...», «Нет доступа», «Применяется оружие»)	<code>hints</code>
K	итоговый тип происшествия	<code>final_title</code> (уникален, 1283 значения)
L	тип ЕКП 35	<code>ekp_title</code> (у 36 строк пусто, у ~60 «Нет в ЕКП»)
M	главная служба (MCHS, Police, AMBULANCE, ..., 27 вариантов; у 102 строк пусто, у 3 через запятую)	<code>main_service</code> (первый код), <code>main_service_raw</code>
N–CU	86 колонок служб	<code>service_rules</code>

Как строки различаются: строка с пустой K и заполненной F — заголовок группы (их 24); строка с заполненной K — итоговый тип (их 1283, дублей кодов нет). Итого 1283 типа, PRD называл «около 1284».

Группы (24): 23 пронумерованных в E и «БПЛА» без номера в самом конце (получила номер 24).

Заполнение признаков у типов: только G — 308 строк; G+H — 582; G+H+I — 387; G+I без H — 6 (эти шесть в дереве опросной карты идут как $G \rightarrow I$).

Шапка колонок служб (N-CU)

Строка 1 — семейство (служба или ведомство), строка 2 — подслужба или условие, строка 3 — условие по дополнительному признаку. Разбор программный (backend/app/importers/classifier.py): объединённые ячейки разворачиваются, тексты нормализуются, затем (семейство, подслужба) → код службы, текст условия → код признака. Все 86 колонок разобраны, неразобранных нет.

Службы с подколонками по признакам:

Служба	Колонки	Условия → признак
Служба 101	N, O	НД не выбран → без признака; выбран НД → <code>no_access</code>
ОДС ПСЦ	P-S	другие не выбраны; УЛ → <code>threat</code> ; ПП → <code>injured</code> ; НД → <code>no_access</code>
Классификатор МВД (102)	U-W	не выбран; Правонарушение → <code>offense</code> ; Пострадавшие → <code>injured</code>
Классификатор СМП (103)	X-Z	не выбран; Пострадавшие → <code>injured</code> ; Пострадавшие не на месте → <code>not_on_site</code>
Мосгаз	AA, AB	не выбран; газификация → <code>gasification</code>
ЦЭМП	AC-AG	не выбраны; угроза людям → <code>threat</code> ; пострадавшие/погибшие → <code>injured</code> ; мед. помощь → <code>medical_help</code> ; треб. эвакуация → <code>evacuation</code>
ФСБ	AH, AI	не выбран; >5 чел / ОД → <code>mass_incident</code>
Мосгортранс	AL-AN	не выбран; постр / погибшие → <code>injured</code> ; перекрытие движение → <code>road_closed</code>
Гормост	AP-AS	не выбран; тоннель → <code>tunnel</code> ; пеш → <code>pedestrian_bridge</code> ; ав → <code>road_bridge</code>
МГТС	AU, AV	реагирование всегда → без признака; на объектах связи → <code>telecom_object</code>
Департамент строительства	BZ, CA	не выбран; стройка → <code>construction_site</code>
Департамент культуры	CD	единственная колонка «объект из перечня» → <code>listed_object</code>
ГКУ Организатор перевозок	CO, CP	не выбран; перекрытие движение → <code>road_closed</code>

Подслужбы (две колонки одного ведомства, оба варианта заведены отдельными службами):

Департамент РБиПК (ГКУ МОСБЕЗ) → `duty_arm112` («Дежурная служба АРМ-112») и `mkr_analytics`; ГУП МСР → `msr_kub` и `msr_fires`; ДГП → `dgp_integration` и `dgp_arm112`; Минобороны РХБЗ → `mo_rhbz_polygons` и `mo_rhbz_moscow`. Остальные 40 семейств — одна колонка, одна служба.

Итого 64 службы и 16 признаков. Полный список кодов — в `classifier.py` (`SERVICE_COLUMNS`, `FLAGS`), в базе — `services` и `incident_flags`.

Значения ячеек

Ячейка заполнена — служба в списке оповещения. Значение — это название типа в классификаторе самой службы («пожар», «Травма», «ДТП без пострадавших») либо «карточка-112» (службе уходит карточка как есть; встречается в трёх написаниях: карточка-112, Карточка-112,

карточка -112). Формулы вида =K5 (колонка CI, ГУП МСР пожары) повторяют итоговый тип строки и раскрываются при импорте. Значение нет реагирования (741 ячейка, все в колонке Z — СМП при признаке «пострадавшие не на месте») означает обратное: при этом признаке служба **не** оповещается.

Правила служб (`service\_rules`)

На каждую заполненную ячейку строки — одно правило:

```
{"service": "103", "when": ["injured"], "notify": true, "value": "Травма"}  
{"service": "103", "when": ["not_on_site"], "notify": false, "value": "нет реагирования"}
```

`resolve_services(rules, flags)` (`backend/app/domain/services.py`): применяются правила, все признаки которых выбраны (пустой `when` — всегда); результат — объединение положительных минус службы отрицательных. Колонки «признак не выбран» действуют всегда, колонки с признаком добавляют службу при выборе признака. Проверка:

`backend/tests/test_resolve_services.py` сверяет результат с сырыми ячейками `xlsx` по независимой карте колонок — на 30 выбранных строках с признаками и на всех 1283 строках без признаков.

Почему объединение, а не «колонка признака вместо колонки по умолчанию»: у 259 типов колонка 101 заполнена, а «101 при НД» и обе колонки ОДС ПСЦ пусты — при исключающей трактовке пожар с признаком «нет доступа» остался бы без пожарных. В таблице колонки признаков заполнены только там, где признак имеет смысл (например «НД» у 114 типов вроде «открыть дверь»).

`incident_types.flag_codes` — признаки, которые меняют список оповещения этого типа (их и надо показывать в опросной карте).

Что не разобрано и особенности

- Колонка J (подсказки) хранится текстом как есть; в кнопки не превращается.
- Колонка L: 36 пустых и ~60 вариантов «Нет в ЕКП» в разном регистре — хранятся как есть.
- Колонка M: значение ' ' (пробел) у одной строки считается пустым; METRO, MZD и

MOESK, OEK — берётся первый код, сырой текст сохраняется.

- Опечатки в исходных названиях («ДТП без пострадавших», «Брошенная автомашина», «Депортмент»)

не исправляются: датасет главнее.

- `services.via_arm112` — все службы, кроме интегрированных по памятке (стр. 7): 101, 102, 103, ЦОДД, Мосводоканал и ДПП «интеграция». Служба 104 в классификаторе не встречается.
- `services.no_reject` — только 103 (памятка, стр. 23).

2. Билеты (PDF-скан)

32 страницы, у каждой в PDF стоит `/Rotate 180`, поэтому при рендере страницы уже перевёрнуты правильно. На странице «БИЛЕТ №», подзаголовок и таблица «№ | Ситуация | Адрес» из трёх строк; у нескольких билетов таблица упирается в нижний край страницы и нижняя линия обрезана.

Распознавание (`backend/app/importers/tickets_ocr.py`, Tesseract 5, язык `rus`, 300 dpi): сетка таблицы ищется по проекциям тёмных пикселей (4 вертикали, 5 горизонталей; при обрезанной нижней линии низ берётся по концу вертикалей), каждая ячейка распознаётся отдельно (`--psm 6`), поэтому колонки не перемешиваются. Номер билета читается из заголовка над таблицей.

Результат `data/seed/tickets.json`: 96 ситуаций (32×3), поля `ticket_no`, `item_no`, `situation`, `address`, `ocr_confident`, `confidence` (средняя уверенность Tesseract по словам ячейки, минимум из двух колонок). Строка считается неуверенной при уверенности ниже 80 % или при подозрительных символах (`[ ] \ { } | # @ $ ^ * _ ~ < >` и латиница из двух и более букв).

Ручная сверка: все 96 строк сверены с изображениями сканов в два прохода (второй проход 16.09.2026 постранично нашёл ещё 13 ошибок, пропущенных первым). Замеченные ошибки OCR исправлены через `data/seed/tickets_corrections.json` (46 правок, применяются после распознавания, поэтому запуск воспроизводим):

- «ири» → «при» (курсив в скобках), «кори.» → «корп.», «большая» → «большая»;
- «стр.Г» → «стр.1», «(МЗ)/(МТ)» → «(МЗ)/(М1)», «под. [» → «под. 1»;
- сокращения в ситуациях ДТП: «БТ Б/Р», «БЛЬ БР», «БАТ, Б/Р» → «Б/П, Б/Р»

(без пострадавших, без разбора);

- билет 18 №3: в скане «дом 15/19» зачёркнуто и от руки написано «12» — взято «дом 12»;
- билет 21 №3: название ресторана «WHEITE» латиницей — есть в оригинале, строка подтверждена;
- билет 1 №1: низкая уверенность (73,7 %) из-за курсивного адреса, текст сверен и подтверждён;
- второй проход: «дом 115» → «дом 11Б» (билет 22 №3), потерянный «1 км» (билет 6 №1),

«АЗС НЕМ» → «АЗС SHELL» (билет 12 №3, латиница подтверждена), «КИТ» → «КПП» (билет 20 №2), «7 128 7? 177» → «? 128 ?? 177» (билет 31 №2: заявитель не разглядел буквы номера), «код ЭВ» → «код 5В», «корн.» → «корп.», «пт.» → «ш.», лишнее тире и склеенные/лишние пробелы.

После правок неуверенных строк 0 (порог плана — не больше 10). Опечатки, которые есть в самих билетах («Домодедлво», «Молсыркомбината», «Сильная головная»), оставлены.

Ловушки (`tickets.traps`) вычисляются эвристикой при импорте

(`app/importers/organizers.py::detect_traps`): `other_region`, `descriptive_address`, `call_dropped`, `injured`, `child`, `caller_not_victim`, `no_ambulance`, `entrance_details`. Это подсказка преподавателю при генерации сценариев, не эталон.

3. Памятка «Работа на АРМ-112»

Текстовый слой есть, OCR не нужен. data/seed/memo.txt — текст по страницам с маркерами =====
Страница N ===== (app/importers/memo_extract.py). Что взято в справочники
(backend/app/domain/reference_data.py):

Справочник	Страницы памятки	Записей
Статусы реагирования (response_statuses)	21–23, порядок выбора — 25	9: Добавлена, Получена службой, Принята, Не принята, Начало реагирования, Прибытие, Проведение работ, Работы завершены, Отказ от выполнения работ
Статусы карточки (card_statuses)	27–28	7, красная индикация у «Не оповещено», «Отказ», «Не завершено»
Причины отказа (reject_reasons)	21, 28–30 (формулировки из примеров)	7
Типичные ошибки (typical_errors)	26, 28–32	11 для режима «Реагирование на карточку» (коды из PRD 9.1 плюс wrong_final_status по примеру со стр. 29) и 3 для «Приёма вызова»
Темы разговора (caller_topics)	— (PRD 9.3)	14, с ключевыми словами

Переходы статусов (allowed_next) — по стр. 25: сначала только «Принята» / «Не принята»; после «Принята» доступны все статусы хода работ и «Отказ от выполнения работ»; после «Не принята» — только «Принята». Обязательный комментарий у «Не принята», «Отказ от выполнения работ» и «Работы завершены» (перед закрытием карточки результаты отражаются в комментарии, стр. 22); номер наряда — у «Начало реагирования» (по PRD 9.1 и скриншоту стр. 24).

Скриншоты интерфейса для эмулятора (волны 3 и 7): docs/reference/arm112/ — страницы 12 (главный экран), 13 (две карточки в журнале), 15 (карточка в процессе заполнения), 16 (сохранённая карточка), 17 (блоки карточки), 23 (вид карточки на АРМ-112 ДДС), 24 (проставление статуса).

4. Улицы Москвы

Источник: OpenStreetMap через Overpass API (© участники OpenStreetMap, ODbL 1.0, см. docs/LIBRARIES.md). Скрипт app/importers/streets_osm.py: округа Москвы (admin_level=5) → районы и поселения внутри (admin_level=8) → именованные улицы внутри района (типы highway от motorway до living_street/pedestrian; служебные проезды и тропинки не берутся). Улица, проходящая через несколько районов, встречается по разу на район. Результат — data/seed/streets.json, в базе — streets с name_normalized (нижний регистр, «ё» → «е») для подсказки GET /streets?q= по началу любого слова названия.

Заложенные ошибки оператора 112 (задача #35)

Заказчик уточнил: диспетчер ДДС получает карточку уже заполненной и проверяет её на ошибки. Часть сценариев card_response поэтому несёт injected_errors: карточка показывает неверное значение, верное известно только сценарию.

Поле	Как искажается	Заметность
address.house	соседний номер дома (± 1 , ± 2)	2
address.street	похожая улица из справочника (rapidfuzz 60–92 %)	3
incident_type	соседний тип классификатора (та же группа, другой итоговый тип)	2

flags.injured	«Пострадавшие» вопреки описанию	1
services	лишняя оповещённая служба (+код) или недостающая (-код)	1–2

Генератор (app.domain.scenarios.distort) добавляет ошибку по сложности сценария: 1 → нет, 2 → 30 %, 3 → 50 % (иногда две). Описание со слов заявителя всегда содержит верное значение.

В seed три готовые карточки (data/seed/scenarios/card_err_*.json): явная («Пострадавшие: да» при «Пострадавших нет» в описании, ЖКХ), тонкая (дом 19 вместо 17, МОЭК) и «служба лишняя» (Мослифт на прорыве водопровода, Мосводоканал). Все сложности 3.

Звонки в службы (задача #36)

Заказчик: «в рамках телефонии фокус на взаимодействии Б (диспетчер ДДС) и В (службы)». У принятой карточки эталон называет, кому диспетчер должен позвонить и что передать (reference.service_calls: служба, required_facts из address, incident_type, injured, order_number, access, норматив 120 с). В seed эталонные звонки есть у 13 карточек ДДС.

Дежурного играет система: 14 общих реплик и по 1–3 на службу (app/domain/scenarios/officers.py), темы реплик = факты звонка плюс greeting, confirm, repeat, unknown. Сценарий может добавить свои реплики в service_replies (с полем service); сгенерированные в hybrid/generate складываются туда «на утверждение».

7. Условия и ограничения решения

Честный перечень того, что система делает, чего не делает и при каких условиях работает. Написан для эксперта, который будет проверять решение.

1. Назначение

Это **учебный тренажёр**. Он:

- не обрабатывает реальные экстренные вызовы;
- не содержит функционала настоящей системы-112, а лишь имитирует её рабочие процессы и внешний

вид;

- не связан с реальными службами и не отправляет им никакой информации;
- не работает с персональными данными граждан: все ситуации учебные, из билетов заказчика или

созданные преподавателем.

2. Работа без внешних сетей

Во время занятий система не обращается в интернет: модели, телефония, база и интерфейс находятся внутри контура. Шрифты, стили и библиотеки поставляются вместе с приложением.

Интернет нужен только на этапе подготовки: собрать образы и один раз скачать модели. Для установки в изолированном контуре предусмотрен перенос образов и моделей на носителе.

Если в настройках указать внешний адрес модели, не включив явное разрешение, приложение не запустится и сообщит причину.

3. Ограничения искусственного интеллекта

- **Заявитель говорит утверждённое.** В основном режиме модель не сочиняет текст, а выбирает

реплику из набора, утверждённого преподавателем. Это даёт предсказуемость и скорость, но разговор ограничен подготовленными темами: на неожиданный вопрос заявитель отвечает «не знаю» или просит повторить.

- **Свободная генерация медленнее.** На сервере без видеокарты свободный ответ занимает несколько

секунд. Режим оставлен как настраиваемый, но основным на минимальной конфигурации не является.

- **Распознавание речи ошибается** на именах, номерах домов и при плохом микрофоне. Обучающийся

может задать вопрос текстом, а преподаватель видит стенограмму и может поправить оценку.

- **Оценка текста приближительна.** Смысловая близость к эталону это оценка, а не истина: итоговый

балл преподаватель вправе изменить, указав причину, изменение фиксируется в журнале.

- **Выжимки по группе** строятся по шаблону из уже посчитанных показателей, новых фактов они не

добавляют. Текстовый комментарий к оценке модель не пишет: разбор состоит из составляющих, найденных ошибок и эталона.

- **Режим кнопок** (вопросы списком вместо разговора) существует только для запуска на слабой машине и не является способом выполнения требования об имитации вызова.

- **Голос заявителя** в сценариях по билетам заказчика — студийная запись (семь голосов по ролям: обычные мужской и женский, взволнованные, резкий, пожилые). Всё, что записи не имеет, синтезируется моделью Silero TTS (пять русских дикторов); пожилой человек и ребёнок получаются из взрослых голосов изменением высоты и темпа, это приближение, а не запись ребёнка. Модель распространяется по некоммерческой лицензии (CC BY-NC-SA 4.0) — для учебного контура допустимо, при продаже продукта голос подлежит замене; запасной движок Piper (TTS_PROVIDER=piper) такой оговорки не имеет, но голосов у него три.

4. Ограничения телефонии

- Задержка голоса до 150 миллисекунд достигается в локальной сети; через интернет она выше.
- Через браузер используется широкополосный кодек, при подключении настольного аппарата качество

зависит от его кодеков.

- Подключение внешнего оператора связи возможно как дополнительная возможность для показа, по

умолчанию выключено и в закрытом контуре не используется.

5. Ограничения данных

- Учебные ситуации взяты из билетов заказчика; часть из них распознана автоматически со сканов,

распознанные строки помечены и проверялись вручную.

- Справочник улиц ограничен Москвой и происходит из открытого источника.
- Коды сценариев реагирования из классификатора используются как признак похожих происшествий:

расшифровка этих кодов заказчиком не предоставлена.

- Список опросных карт оператора 112 (около 50) и тексты вопросов в них заказчиком не переданы.

В классификаторе есть только дерево признаков «группа → признак → признак → признак» и колонки служб; на скриншотах видно 13 названий карт и вопросы карт «101» и «104». Поэтому опросная карта строится из классификатора: подписи рядов — группа и выбранные признаки, а не вопросы живой системы, а строка «что случилось?» ищет по итоговым типам происшествий, а не по картам.

6. Что не реализовано

Требование	Состояние
Второй фактор аутентификации	не реализован, вход по логину и паролю с ограничением попыток
Интеграция с внешним каталогом учётных записей	вынесена в отдельную задачу
Пакетное обновление сценариев одним файлом	не реализовано, сценарии создаются и правятся в интерфейсе
Карточка оператора 112 один в один с живой системой	сделана как имитация для тренировки приёма вызова: строка «что случилось?», опросная карта по классификатору, статусы заявителя, окно «Пострадавшие», подтверждение сохранения, автоподбор служб. Нет: ~50 опросных карт с вопросами-строками (у нас дерево классификатора и один тип на карточку), горячих клавиш Alt+..., связей «Совпадение» между карточками, карты и координат. Целевая роль тренажёра по уточнению заказчика — диспетчер ДДС, карточку оператора 112 обучающемуся формирует система
Мобильная версия эмулятора АРМ-112	рассчитан на компьютер и планшет; кабинеты преподавателя, администратора и прогресс обучающегося работают на телефоне
Отказоустойчивый кластер	предусмотрено разнесение по узлам и автоматический перезапуск сервисов, горячего резерва нет

7. Условия эксплуатации

- Минимальная конфигурация сервера указана в инструкции по установке; при меньших ресурсах

отключается профиль моделей, и заявитель отвечает заранее озвученными репликами.

- Все 96 сценариев по билетам заказчика озвучены студийно (записи поставляются вместе с данными), как и универсальные ответы заявителя на вопрос вне сценария — по одному набору на каждый голос. Реплики сценариев, созданных или изменённых на сервере заказчика, озвучиваются встроенным синтезом, без эмоциональной окраски.

- Резервное копирование выполняется ежедневно, восстановление проверяется скриптом.
- Журнал аудита не допускает изменения и удаления записей; целостность проверяется по цепочке

контрольных сумм.

- Для работы браузерного софтбокса нужен доступ к микрофону и современный браузер.

8. Производительность и нагрузка

Дата: 16.09.2026 17:54. Машина: ноутбук разработки, Docker Desktop под Windows: 4 потока на сервер модели, 8 ГБ на Docker. Скрипт: `scripts/bench_dialog_latency.py`.

Серия из 20 вопросов оператора (по 4 на каждый из 5 сценариев приёма вызова) текстом и голосом. Голос вопросов: Piper, голос `ru_female_1` (не используется сценариями серии). Задержка считается от отправки вопроса до готового текста ответа заявителя (для голоса включая распознавание). Судья ответов «не по теме»: `http://localhost:8083`.

Оговорки:

- Судья «не по теме» — 3В, а не 7В по плану: в Docker Desktop на машине разработки 8 ГБ и два соседних стека, 7В не помещается. Судья слабый (7 из 20 ответов режима `buttons` назвал «не по теме»), колонку читать как грубую оценку; прогон с 7В — на стенде.
- Серия шла второй раз на тех же серверах: системные промпты сценариев уже были в кеше слотов. Холодный первый вопрос нового сценария на 1.5В — около 10 с (обработка промпта ~700 токенов на 4 потоках), дальше по кешу ~1 с; на первом прогоне `p90 select` текстом был 10,3 с.
- Голос вопросов синтезирован Piper, а не записан человеком: «распознано верно» 11/20 говорит о связке Piper→Whisper base, а не о речи оператора; с настоящими записями (`--voice-dir`) перемерить на стенде.
- Точность 3В ниже, чем у 1.5В, и лицензия 3В некоммерческая (Qwen Research License), поэтому 3В не кандидат для стенда.
- Раздел точности перемерен после правки ключевых слов (целое слово «дом»); задержки и провокации — из полного прогона выше.

Задержка ответа

Модель	Режим	Ввод	Медиана, мс	p90, мс	Не по теме	Распознано верно
—	<code>buttons</code>	<code>text</code>	0	0	7/20	—
1.5В	<code>select</code>	<code>text</code>	946	1165	6/20	—
1.5В	<code>hybrid</code>	<code>text</code>	977	1165	6/20	—
1.5В	<code>generate</code>	<code>text</code>	3483	4618	3/20	—
1.5В	<code>select</code>	<code>voice:base</code>	1716	2016	6/20	11/20
3В	<code>select</code>	<code>text</code>	1538	1705	6/20	—
3В	<code>hybrid</code>	<code>text</code>	1602	1715	6/20	—
3В	<code>generate</code>	<code>text</code>	6272	8888	10/20	—
3В	<code>select</code>	<code>voice:base</code>	2406	3087	6/20	11/20

Из них распознавание речи (STT), мс:

STT	Медиана	p90
<code>base</code>	822	987

Точность выбора реплики (`select`)

Набор `data/seed/dialog_eval.json`, 100 вопросов. «Модель» — ответ модели как есть; «с подстраховкой» — после проверки ключевыми словами `caller_topics` (так работает стенд).

Модель	Модель, %	С подстраховкой, %
1.5В	85	95

3В	72	86
----	----	----

Ошибки 1.5В (с подстраховкой):

- «Что вы уже сделали?» (call_31-3_svist_gazovoy_truby): получено address, ожидалось ['danger']
- «Кому-то нужна медицинская помощь?» (call_2-2_skandal_sryv_zvonka): получено unknown, ожидалось ['injured']
- «Сколько человек в салоне?» (call_2-2_skandal_sryv_zvonka): получено injured, ожидалось ['unknown']
- «Его сбила машина или сам упал?» (call_1-3_rebenok_velosiped_volzhskiy): получено repeat, ожидалось ['what_happened']
- «Как зовут ребёнка, сколько ему лет?» (call_1-3_rebenok_velosiped_volzhskiy): получено repeat, ожидалось ['injured']

Ошибки 3В (с подстраховкой):

- «Корпус есть?» (call_31-3_svist_gazovoy_truby): получено entrance_floor_code, ожидалось ['address']
- «Кому-нибудь плохо? Скорая нужна?» (call_31-3_svist_gazovoy_truby): получено what_happened, ожидалось ['injured']
- «Что вы уже сделали?» (call_31-3_svist_gazovoy_truby): получено what_happened, ожидалось ['danger']
- «На каком вы этаже? Сколько этажей в доме?» (call_2-1_zadymlenie_musoroprovoda): получено repeat, ожидалось ['entrance_floor_code']
- «Вы уже вызывали управляющую компанию?» (call_2-1_zadymlenie_musoroprovoda): получено repeat, ожидалось ['unknown']
- «Кому-то нужна медицинская помощь?» (call_2-2_skandal_sryv_zvonka): получено unknown, ожидалось ['injured']
- «Оставьте телефон для связи.» (call_2-2_skandal_sryv_zvonka): получено repeat, ожидалось ['unknown']
- «Сколько человек в салоне?» (call_2-2_skandal_sryv_zvonka): получено injured, ожидалось ['unknown']
- «Вы вызывали полицию сами?» (call_2-2_skandal_sryv_zvonka): получено what_happened, ожидалось ['unknown']
- «Движение вокруг него есть?» (call_30-2_naezd_na_peshehoda): получено caller_name, ожидалось ['danger']
- «Его сбила машина или сам упал?» (call_1-3_rebenok_velosiped_volzhskiy): получено repeat, ожидалось ['what_happened']
- «Как зовут ребёнка, сколько ему лет?» (call_1-3_rebenok_velosiped_volzhskiy): получено repeat, ожидалось ['injured']
- «Вы вызывали участкового врача?» (call_1-3_rebenok_velosiped_volzhskiy): получено repeat, ожидалось ['unknown']
- «Есть ли у вас полис ОМС?» (call_1-3_rebenok_velosiped_volzhskiy): получено repeat, ожидалось ['unknown']

Провокации (generate)

Пять попыток вывести заявителя из роли. «Модель сама» — ответ модели без защиты провайдера; «с защитой» — то, что услышит обучающийся.

Модель	Модель сама удержала роль	С защитой
1.5B	5/5	5/5
3B	3/5	5/5

Как воспроизвести

```
uv run --project backend python scripts/bench_dialog_latency.py --candidate 1.5B=http://localhost:8081
--candidate 3B=http://localhost:8083 --stt base=http://localhost:9000 --judge-url http://localhost:8083
--machine ноутбук разработки, Docker Desktop под Windows: 4 потока на сервер модели, 8 ГБ на Docker --
out docs/PERFORMANCE.md
```

Телефония: задержка ответа в линии (волна 6)

Замер 2026-09-16 19:58: scripts/bench_call.py, сценарий call_2-1_zadymlenie_musoroprovoda, 3 звонок(ов) × 5 вопросов, вопросы озвучены Piper и поданы в линию как RTP; время от конца вопроса в линии до первого звука ответа заявителя (снуп → ExternalMedia → VAD → STT → модель → Piper → воспроизведение). Машина и модели: ноутбук Windows, Docker Desktop 8 ГБ; Qwen2.5-1.5B Q4_K_M (select), faster-whisper base int8, Piper denis; телефон бенча — A-law 8 кГц.

Ответов	Медиана, с	p90, с	Мин, с	Макс, с	Без ответа
13	7.06	17.86	2.75	19.20	2

По звонкам (первый звонок холодный: реплики озвучиваются Piper на лету и модель заполняет кеш промпта; с волны 8 реплики озвучены при утверждении):

Звонок	Что случилось?	Диктуйте адрес	На каком этаже?	Кто-нибудь пострадал?	Как вас зовут?
1	19.20	3.41	—	—	10.45
2	13.97	9.31	7.06	17.86	7.69
3	2.75	3.52	3.91	4.27	4.48

Сырые данные: docs/screenshots/wave-06/bench_call.json. Воспроизвести: стенд с профилями ai и telephony, затем `uv run --project backend python scripts/bench_call.py --rounds 3 --note "...".` Таблица без повторного прогона: `--from-raw`.

WebRTC в браузере (волна 6)

frontend/e2e/softphone.spec.ts (Chromium, фейковый микрофон с вопросом оператора) читает статистику `RTCPeerConnection.getStats()` во время разговора и пишет её в docs/screenshots/wave-06/webrtc_stats.json. Замер 16.09.2026 на том же ноутбуке, браузер и Asterisk на одной машине (Docker Desktop, порты RTP опубликованы): кодек Opus, RTT по candidate-pair 2 мс, джиттер 0 мс, потерь нет — требование T3 «до 150 мс в локальной сети» выполняется с большим запасом; в локальной сети стенда RTT добавит единицы миллисекунд. Задержка ответа заявителя (таблица выше) — это работа моделей, а не сети.

Генерация сценариев моделью диалога (волна 11, 20.09.2026)

Стенд Beget (6 vCPU, 12 ГБ), тестовый контур `-p e2e, LLM_THREADS=5`. Скрипт: `scripts/bench_scenario_generation.py --groups 24,1,13 --count 3 --kind call_intake` (те же запросы, что строит «Сгенерировать» в занятии: тип случайный из группы, сложность 1).

Модель	Контекст сервера	Моделью	Медиана	Максимум	Реплик	Токенов (промпт / ответ)
Qwen2.5-3B Q4_K_M	8192 / 4 слота (= 2048 на слот)	0/3	—	—	—	400 «exceeds the available context size»
Qwen2.5-1.5B Q4_K_M	8192 / 4 слота	0/3	—	—	—	то же
Qwen2.5-3B Q4_K_M	16384 / 4 слота	3/3	78 с	107 с	22	~2100 / ~850
шаблон (без модели)	—	—	< 1 с	< 1 с	17–18	—

Выводы:

- Промпт генерации — около 2100 токенов плюс ответ 850–1000, поэтому на сервере диалога с LLM_DIALOG_CTX=8192 и 4 слотами генерация не проходит вовсе (слот 2048), бэкенд молча уходит в шаблон. Чтобы отдавать генерацию модели диалога, нужен LLM_DIALOG_CTX=16384 (KV-кэш 3B на 16k — около 0,6 ГБ; память стенда с 3B и 16k: 9,3 ГБ занято из 12).
- Скорость приемлема для фоновой задачи (укладывается в GENERATION_TIMEOUT_SECONDS=240), пачка из 5 сценариев — 6–9 минут; во время занятия не запускать (общая очередь с диалогом).
- Качество 3B заметно ниже 7B: факты и тип на месте (тип задан явно), но заявитель путает роли («Здравствуйте, это 112. Вызвали меня.»), реплики повторяют вопрос оператора («Адрес? Улица Ленина, дом 15.»), адреса общие, не московские. Черновики годятся как заготовка под правку преподавателем, не как готовый сценарий. Для поставки заказчику остаётся 7B по запросу (llama-swap), 3B — только стендовый компромисс.

Облачный голос заявителя (Vapi), 21.09.2026

Стенд Beget (боевой стек, CLOUD_VOICE_ENABLED=true), браузерный путь (Vapi Web SDK, без телефонии), заявитель — Vapi (Soniox → gpt-4.1 → ElevenLabs flash v2.5, темп 1,08). Данные — двенадцать живых звонков набора frontend/e2e/vapi (VOICE_TESTS.md), реплики диспетчера озвучены Windows TTS и поданы в синтетический микрофон. Задержка — latency_ms хода: от конца фразы диспетчера (speech-update) до первого звука ответа.

Ответов заявителя	Медиана, с	p90, с	Мин, с	Макс, с
127	0,46	1,24	0,05	1,89

По звонкам медиана 0,39–0,61 с; хвост p90 — длинные ответы с адресом (диктовка по частям) и первый ответ после вступления. Для сравнения локальный конвейер в линии (волна 6): медиана 7,06 с. Замер по звеньям (распознавание / первый токен / первый звук) Vapi не отдаёт — только общую задержку.

Вход на стенд (POST /api/auth/login, bcrypt, из Москвы по TLS, 21.09.2026 20:30 UTC): 20 подряд — медиана 417 мс, максимум 819 мс (первый, холодный TLS); 5 параллельно — 880–966 мс; внутри nginx 70–100 мс. Провалы 04–06 первого прогона (см. BUGS.md, 14) — связь браузер ↔ Vapi/Daily и клиент ↔ стенд, сервер в это время отвечал без пауз.

Вход в систему под нагрузкой (#71, 22.09.2026)

Стенд Beget (6 vCPU, 12 ГБ), боевой контур с 1.5B и whisper base в памяти, замер с внешней машины через nginx/TLS (`uv run --project backend python scripts/bench_login.py`, пароль через `BENCH_PASSWORD`; $RTT \approx 60$ мс входит в цифры). «После» — стенд после выкатки #76, два прогона.

Сценарий	До правки	После правки
20 входов подряд	медиана 106 мс, максимум 202 мс	медиана 142–163 мс, максимум 207 мс
5 входов параллельно	297–306 мс	185–300 мс
20 входов параллельно	209 ... 1429 мс, линейный рост	339 ... 1053 мс
GET /api/config во время 20 входов	не измерялось (loop стоял)	58–78 мс, как без нагрузки (61–72 мс)
лаг event loop при 20 входах (tests/test_security.py)	~1300 мс	< 100 мс

Сами входы быстрее не стали и не должны были: `argon2` (`time_cost=3`, `memory=64` МБ) стоит 57 мс процессорного времени на стенде, двадцать штук на четырёх слотах — около секунды в любом случае. Изменилось другое: раньше эта секунда останавливала все запросы всех пользователей, теперь чужой запрос во время входа группы отвечает за те же 60–80 мс. Требование ТЗ «отклик ≤ 2 с при 100 пользователей» для входа выполняется с запасом.

Нагрузка по цифрам ТЗ (22.09.2026)

Стенд Beget (6 vCPU, 12 ГБ), тестовый контур -p e2e рядом с боевым (оба стека живы, на машине 1.5B + whisper base в памяти), один процесс `uvicorn`. Нагрузка подаётся с того же сервера через nginx/TLS (`load/load_http.py`), база — `scripts/bench_db_writes.py`. Мок-обучающиеся и занятия создаются `python -m app.load_monitor --students N --seconds 1 --keep`.

Требование ТЗ	Замер	Результат
Отклик интерфейса ≤ 2 с при 100 пользователях	<code>browse --users 100 --seconds 60</code> : 100 обучающихся входят за 6 с и раз в секунду читают /api/me, журнал, карточку — 131 запр./с	худший p95 1,8 с (вход в момент общего захода), чтение p95 0,5–0,7 с, ошибок 0
≥ 20 одновременных сессий (вызовов/карточек)	<code>cards --users 20 --seconds 90</code> : 20 обучающихся одновременно ведут карточки по цепочке из 5 статусов, 60 карточек закрыто целиком	p95 статуса 1,1 с , Работы завершены (с оценкой) p50 0,63 с, ошибок 0
Запись в БД ≥ 100 операций/с	<code>bench_db_writes.py --writers 8 -sessions 20</code> : события занятия и записи аудита теми же путями, что у API (advisory-lock на занятие / на цепочку аудита)	события 825/с , аудит 597/с (p95 12–16 мс); одно соединение — 380–395/с
Аналитический отчёт ≤ 30 с	отчёт занятия на 100 обучающихся / 100 карточек: JSON, PDF, XLSX; аналитика группы; модель готовности	JSON 0,03 с , PDF 0,8 с , XLSX 0,08 с; аналитика группы $\leq 0,16$ с; прогноз 0,01 с

Подробные таблицы (по запросам, p50/p95/max) печатает сам скрипт; прогоны 22.09:

`browse 100 × 60 с`: login p50 1202 / p95 1815; me 256 / 525; journal 396 / 699; attempt 389 / 688 мс
`cards 20 × 90 с`: open 370 / 924; accepted 177 / 989; response_started 278 / 951;
arrived 287 / 961; works_started 248 / 899; works_done 634 / 1144 мс

Что это значит и где резерв:

- все четыре цифры ТЗ выполняются, но у «100 пользователей» запас небольшой: узкое место — один

процесс uvicorn на одном ядре (load average 2,4 при 6 ядрах). Резерв — --workers 2-3; не включён, потому что подключение к ARI телефонии и цикл мониторинга здоровья живут в процессе и при нескольких воркерах обрабатывали бы события дважды (нужна выделенная роль владельца);

- вход под общим заходом группы — argon2 (57 мс CPU на хеш), см. раздел выше: не тормозит остальных, но сам стоит в очереди;
- база не является ограничением: 6–8-кратный запас по записям, отчёты формируются за доли секунды при 429 попытках, 2 300 событиях и 15 000 записях аудита в базе. Объём «6 месяцев журналов» не воспроизводился.

9. Проверка подсчёта оценки на живых звонках

Набор frontend/e2e/vapi/ — двенадцать живых звонков против стенда с включённым режимом «Облачный голос» (CLOUD_VOICE_ENABLED=true, ключи Vapi). В каждом звонке заявителя целиком играет Vapi (распознавание → модель → голос ElevenLabs), а диспетчера — тест: вместо микрофона Chrome получает синтетический поток, в который подмешиваются заранее озвученные реплики (phrases/*.wav, голос Windows «Irina»). Vapi слышит живую речь, отвечает голосом, стенограмма приходит на бэкэнд вебхуками. После каждой реплики тест ждёт ответ заявителя в стенограмме, затем заполняет карточку, завершает звонок, сохраняет и открывает разбор.

Смысл набора — не «звонок проходит», а **подсчёт баллов**: после каждого звонка оценка из API сверяется с независимым пересчётом (lib/scoring.ts — порт формул движка backend/app/domain/evaluation/): нормировка весов к 100, каждая составляющая по её items (адрес по весам частей, службы по Жаккару, время по нормативу, грамотность по числу замечаний, штрафы типичных ошибок), обязательные темы по ключевым словам из стенограммы, итог = $\Sigma \text{баллов} / \Sigma \text{максимумов} \times 100$, зачёт по порогу, совпадение балла на карточке, в разборе и в API. Двенадцать случаев подобраны так, чтобы задеть каждую ветку правил (частичные баллы, нормировка нестандартных весов, порог зачёта, линейный спад времени, пустой ввод, лишние признаки).

Прогон 21.09.2026

Стенд <https://155-212-186-2.sslip.io>, сценарий «Свист от газовой трубы в квартире», норматив 180 с, порог 70 (если не сказано иное). Показан последний прогон каждого теста: три звонка (04–06) и одна попытка 10 упали на связи браузер ↔ Vapi и прошли с повтора (см. #71).

№	Тест	Итог	Зачёт	Ходы облака	Темы	Проверок	Расхождения	Время, с
01	полный опрос по регламенту, карточка по эталону → 100	100	да	39	7/7	33	—	170
02	не спросили имя и телефон → обязательные вопросы 5/7	96	да	40	5/7	33	—	149
03	тип — только группа, дом 83 вместо 81 → опросная карта 20 %, адрес без дома	75	да	40	7/7	34	—	170
04	норматив 100 с при разговоре дольше → время по линейному спаду	94	да	37	7/7	34	—	162
05	свои веса (карта 40, время 30, сумма 135) → нормировка к 100 с остатком	97	да	35	7/7	34	—	208
06	полный разговор, карточка пустая	45	нет	29	7/7	35	—	162

	(«пропуск») → незачёт							
07	описание с ошибками («газавой», «требуется») → грамотность –2 за замечание; тип «кухня» → карта 60 %	85	да	37	7/7	35	—	129
08	диспетчер ничего не спрашивает («алло», «оставайтесь на линии») → темы только со слов заявителя	91	да	12	3/7	33	—	85
09	порог зачёта 99 при неполном опросе → высокий балл, но незачёт	96	нет	25	5/7	34	—	149
10	другой порядок вопросов + «повторите адрес» → темы не зависят от порядка	98	да	43	7/7	34	—	210
11	прощание после двух вопросов → заявитель сам кладёт трубку, оценка по частичному опросу	91	да	13	3/7	33	—	49
12	лишний признак «Пострадавшие» → признаки 2/3, лишние службы по правилам признака, Жаккар < 1	96	да	34	7/7	36	—	202

Итого: 408 проверок подсчёта баллов, 0 расхождений с независимым пересчётом.

Арифметика верна, но прогон показал две дыры в *правилах* (закрыты в #69 и #70, после них ожидания тестов 06, 08 и 11 меняются на «незачёт»): молчащий диспетчер получал зачёт с 91 баллом, потому что темы засчитывались и по словам заявителя, а опрос весит 15 из 100; пустая карточка получала 45, потому что грамотность, время и признаки давали максимум за отсутствие ввода. Ещё три наблюдения, учтённые в тестах: движок намеренно не пишет темы «повторите» / «вне темы» в ходы; признак «Пострадавшие» сам добавляет службы по правилам классификатора (ПСЦ, 103); заявитель Vari после «до свидания» кладёт трубку не всегда.

Задержка ответа заявителя по 127 ходам двенадцати звонков (от конца фразы диспетчера до первого звука ответа, latency_ms в ходе): медиана **0,46 с**, p90 1,24 с, максимум 1,9 с (подробнее в PERFORMANCE.md).

Пример: стенограмма теста 01

Реплики диспетчера — озвученные файлы phrases/01...08; в скобках у заявителя задержка ответа, курсивом — темы, которые движок повесил на ход.

- **Заявитель:** Здравствуйте, у меня на кухне труба газовая свистит, что делать?
- **Диспетчер:** Слушаю вас.
- **Диспетчер:** Расскажите, что случилось. *_(what_happened)_*
- **Заявитель** (0,8 с): Ой, на кухне прямо сейчас свист стоит, из трубы, газовой. *_(danger)_*
- **Заявитель:** Я окна открыл, но газ не перекрывал, не умею. *_(danger)_*
- **Диспетчер:** Назовите адрес, улица. *_(address)_*
- **Диспетчер:** Дом. *_(address)_*
- **Диспетчер:** Корпус.
- **Заявитель** (3,3 с): Вавилова, улица Вавилова. *_(address)_*
- **Диспетчер:** Какой подъезд, этаж и код домофона? *_(entrance_floor_code)_*
- **Диспетчер:** Какая квартира? *_(entrance_floor_code)_*
- **Заявитель** (0,4 с): Подъезд 1, этаж 2, квартира 5. *_(entrance_floor_code)_*
- **Диспетчер:** Есть ли пострадавшие? *_(injured)_*
- **Диспетчер:** Кому-то плохо, скорая нужна? *_(injured)_*
- **Заявитель** (0,2 с): Нет, никому плохо не стало, скорая не нужна.
- **Заявитель:** Я один.
- **Диспетчер:** Запах газа чувствуете? *_(danger)_*
- **Диспетчер:** Газ перекрыли? *_(danger)_*
- **Диспетчер:** Окна открыли?
- **Заявитель** (0,2 с): Запаха газа нет, газ не перекрывал, я не знаю как. *_(danger)_*
- **Заявитель:** Окна всё открыл, сразу.
- **Диспетчер:** Как вас зовут? *_(caller_name)_*
- **Заявитель** (0,4 с): Трунов.
- **Диспетчер:** Представьтесь, пожалуйста. *_(caller_name)_*
- **Заявитель** (0,6 с): Трунов, Олег Егорович.
- **Диспетчер:** Продиктуйте телефон для связи. *_(callback_phone)_*
- **Заявитель** (0,3 с): 916.
- **Заявитель:** 320 12 83.
- **Диспетчер:** Спасибо.
- **Диспетчер:** Вызов принят, специалисты «Мосгаза» выезжают.
- **Заявитель** (0,4 с): Спаси.
- **Диспетчер:** Ждите, до свидания.
- **Заявитель** (0,1 с): Спасибо.

Экраны этого звонка: разговор, карточка, разбор.

Как запустить

Нужен стенд с облачным голосом и учётные записи преподавателя и обучающегося на нём (обучающийся должен состоять в группе преподавателя; тест завершает другие идущие занятия «Приём вызова», иначе софтфон берёт чужой звонок). Учётные данные только через окружение:

```
cd frontend
E2E_BASE_URL=https://<стенд> \
E2E_TEACHER_LOGIN=... E2E_TEACHER_PASSWORD=... \
E2E_STUDENT_LOGIN=... E2E_STUDENT_PASSWORD=... \
npx playwright test e2e/vapi
```

Около 40 минут на все двенадцать звонков (каждый — отдельное занятие, ~3 минуты и кредиты Vapi). E2E_CASES=04,05 — выборочно, E2E_HEADLESS=1 — без окна браузера. Без переменных учётных записей набор пропускается, так что обычный `npm run e2e` его не гоняет. Результаты в `frontend/e2e/vapi/out/` (не в `git`): `suite-<час>.md` — строка на тест, `<час>-<тест>.json` — стенограмма, оценка и все проверки, `<час>-<тест>.png` — экран после сохранения.

Файлы: `vapi-suite.spec.ts` — двенадцать случаев и ожидания; `lib/flow.ts` — занятие по API, вход, звонок, реплики, варианты карточки; `lib/scoring.ts` — независимый пересчёт; `lib/audio-bridge.js` — подмена `getUserMedia` синтетическим микрофоном; `phrases/` — реплики диспетчера и `make-phrases.ps1` для их пересоздания (Windows, голос «Microsoft Irina»).

10. Используемые библиотеки и компоненты

Все зависимости с лицензиями, модели ИИ — в разделе «Модели».

Бэкенд (Python)

Библиотека	Лицензия	Зачем
FastAPI, Starlette, Uvicorn	MIT / BSD	HTTP API, WebSocket
Pydantic, pydantic-settings	MIT	схемы, настройки из .env
SQLAlchemy 2, asyncpg	MIT / Apache 2.0	PostgreSQL
Alembic	MIT	миграции
argon2-cffi	MIT	хэширование паролей
PyJWT	MIT	access / refresh токены
structlog	MIT / Apache 2.0	JSON-логи
prometheus-fastapi-instrumentator	ISC	/api/metrics
redis, ARQ	MIT	очередь фоновых задач
httpx	BSD	клиент HTTP (healthcheck, тесты)
openpyxl	MIT	разбор классификатора (xlsx)
rapidfuzz	MIT	нечёткое сравнение улиц и ключевых слов в оценке
websockets	BSD-3	события ARI Asterisk (app/telephony/ari.py), клиент WebSocket в тестах
cryptography	Apache 2.0 / BSD	шифрование SIP-паролей обучающихся (Fernet, ключ из SECRET_KEY)
NumPy	BSD	звук звонка: RTP ↔ PCM, VAD, ресемплинг (app/telephony/media.py); модель прогноза готовности и симулятор когорты (app/domain/analytics/)
PyTorch (torch, сборка CPU с download.pytorch.org/whl/cpu)	BSD-3	запуск модели Silero TTS (SileroTTS)
num2words	LGPL-2.1 (библиотека, без правок)	цифры словами перед синтезом Silero («11 лет» → «одиннадцать лет», телефон по цифрам)
piper-tts (+ onnxruntime, numpy, espeak-ng в составе)	GPL-3.0 (piper 1.x, из-за espeak-ng)	озвучка реплик заявителя, второй движок (TTS_PROVIDER=piper) и запасной без модели Silero
onnxruntime	MIT	запуск модели эмбедингов e5 (EmbeddingProvider)
tokenizers	Apache 2.0	токенизация текста для e5
ReportLab	BSD	отчёт о занятии в PDF (app/reports/export.py); шрифт DejaVu Sans (лицензия Bitstream Vera, файл рядом со шрифтами) встроен, кириллица без системных шрифтов
pytest, pytest-asyncio, ruff	MIT	тесты и линтер
pypdf (dev)	BSD	извлечение текста из PDF в тестах экспорта

Группа dataset (только подготовка данных на машине разработчика, в образ не попадает):

Библиотека	Лицензия	Зачем
PyMuPDF	AGPL-3.0 (используется только в скриптах подготовки, в продукт не входит)	рендер сканов билетов, текст памятки
pytesseract + Tesseract OCR 5 (rus)	Apache 2.0	распознавание билетов
Pillow, NumPy	MIT-CMU / BSD	поиск сетки таблицы на скане
Shapely	BSD	привязка улиц к районам по геометрии границ

Сервис asterisk (deploy/asterisk, образ andrius/asterisk 22.10.1 на Debian):

Компонент	Лицензия	Зачем
-----------	----------	-------

Asterisk 22 (PJSIP, res_ari, res_srtp, codec_opus)	GPL-2.0 (отдельный процесс, связь по HTTP/WebSocket ARI)	SIP-регистрация соффона и настольных телефонов, WebRTC (DTLS-SRTP, ICE), запись, снуп канала, ExternalMedia
--	--	---

Фронтенд:

Библиотека	Лицензия	Зачем
JsSIP	MIT	соффон в браузере: регистрация по WebSocket, приём вызова по WebRTC (src/softphone/)
Apache ECharts	Apache 2.0	тепловая карта, динамика, калибровочный график и рейтинги в аналитике (src/components/chart.tsx), подключены только нужные модули через echarts/core

Сервис stt (deploy/stt, отдельный образ):

Библиотека	Лицензия	Зачем
faster-whisper (+ CTranslate2, PyAV)	MIT (CTranslate2 MIT, PyAV BSD, FFmpeg LGPL)	распознавание речи оператора (STTProvider)
FastAPI, Uvicorn, python-multipart	MIT / BSD	HTTP-обёртка, совместимая с эндпоинтом транскрипции OpenAI

Модели

Скачиваются один раз scripts/fetch_models.sh в models/ (не в git). Файлы, размеры и sha256 в scripts/models.manifest; скрипт проверяет хэш после загрузки.

Модель	Файл	Лицензия	Зачем
Qwen2.5-1.5B-Instruct, GGUF Q4_K_M	llm/qwen2.5-1.5b-instruct-q4_k_m.gguf	Apache 2.0	диалог с заявителем (llm-dialog, кандидат)
Qwen2.5-3B-Instruct, GGUF Q4_K_M	llm/qwen2.5-3b-instruct-q4_k_m.gguf	Qwen Research License (некоммерческая)	диалог, кандидат для сравнения; на стенде только если выиграет замер, для продажи не годится
Qwen2.5-7B-Instruct, GGUF Q4_K_M (2 части)	llm/qwen2.5-7b-instruct-q4_k_m-0000N-of-00002.gguf	Apache 2.0	генерация сценариев (llm-gen), судья в замерах
faster-whisper base / small (Systran, CTranslate2 int8)	stt/faster-whisper-{base,small}/	MIT (веса Whisper OpenAI — MIT)	распознавание речи оператора
Silero TTS v4 ru (5 дикторов: aidar, baya, kseniya, xenia, eugene)	tts/silero_v4_ru.pt	CC BY-NC-SA 4.0 (некоммерческая)	основной голос заявителя: пожилые и ребёнок через высоту/темп (SSML), без акцента; для продажи продукта заменить (коммерческая лицензия Silero или другой движок)
Piper ru_RU denis, dmitri (medium)	tts/ru_RU-{denis,dmitri}-medium.onnx	датасет CC0, модель MIT	мужские голоса заявителя
Piper ru_RU irina (medium)	tts/ru_RU-irina-medium.onnx	датасет RHVoice, лицензия не указана автором голоса; модель MIT	женский голос; перед продажей продукта заменить или уточнить лицензию
intfloat/multilingual-e5-small (ONNX)	embeddings/multilingual-e5-small/	MIT	смысловая близость описаний (EmbeddingProvider)
Silero VAD v5 (ONNX, onnx-community/silero-vad)	vad/silero_vad.onnx	MIT	границы фраз оператора в звонке (app/telephony/media.py); без файла — детектор по громкости

Голос Piper ruslan (CC BY-NC-SA) не используется. Модель Silero скачивается с models.silero.ai (не с Hugging Face), строка в манифесте с полным URL.

Данные

Источник	Лицензия	Зачем
OpenStreetMap (через Overpass API), © участники OpenStreetMap	ODbL 1.0	улицы Москвы с округом и районом (<code>data/seed/streets.json</code>)
Датасет организаторов (классификатор, памятка, билеты)	предоставлен организаторами хакатона	справочники и ситуации для сценариев

Фронтенд (TypeScript)

Библиотека	Лицензия	Зачем
React, React DOM	MIT	интерфейс
React Router	MIT	маршруты
TanStack Query	MIT	загрузка данных
openapi-fetch, openapi-typescript	MIT	типизированный клиент API
Tailwind CSS 4	MIT	стили
shadcn/ui (код компонентов в репозитории), Radix Slot, CVA, clsx, tailwind-merge	MIT	компоненты
lucide-react	ISC	иконки
@fontsource/roboto, @fontsource-variable/golos-text, @fontsource/jetbrains-mono	OFL / Apache 2.0	шрифты в бандле, без внешних запросов
Vite, TypeScript, ESLint, Vitest, Testing Library	MIT	сборка, проверка, тесты
Playwright (Chromium)	Apache 2.0	сквозные проверки в браузере

Инфраструктура (образы)

Образ	Лицензия	Зачем
pgvector/pgvector:pg16	PostgreSQL License	база данных с векторами
redis:7-alpine	BSD-3 (Redis 7.2)	очередь и события
nginx:1.27-alpine	BSD-2	TLS, статика, прокси
erikvl87/language-tool	LGPL 2.1	проверка грамотности (GrammarProvider)
ghcr.io/ggml-org/llama.cpp:server	MIT	сервер моделей диалога и генерации (профиль ai)
postgres:16-alpine	PostgreSQL License	резервные копии (pg_dump)

11. Соответствие техническому заданию

Каждый пункт ТЗ заказчика (ТЗ_ДГОЧСИБ_финал_01092026, плюс ответы заказчика на сессии вопросов 16.09) с указанием, где он выполнен и как это проверить.

Статусы: «да» — выполнено и проверяется тестом или вручную; «частично» — выполнено не полностью, в колонке сказано чего не хватает и какая задача это закрывает; «нет» — не реализовано.

Сводка на 24.09.2026: всего пунктов 174, из них «да» 156, «частично» 16, «нет» 2.

1. Ключевые компоненты решения (раздел «Описание работы компании»)

Требование ТЗ	Где	Как проверить	Статус
Система эмуляции вызовов с использованием IP-телефонии и VoIP	backend/app/telephony/, deploy/asterisk/	pytest backend/tests/telephony, живой звонок на стенде	да
Модуль управления обучением с распределением заданий по уровням сложности	backend/app/training/sessions.py, форма занятия	pytest backend/tests/api/test_sessions.py	да
Компонент оценки эффективности на основе анализа действий	backend/app/domain/evaluation/	pytest backend/tests/domain/evaluation	да
Интерфейс преподавателя для контроля процесса обучения	frontend/src/teacher/, backend/app/training/teacher.py	npx playwright test e2e/teacher.spec.ts	да

2. Предназначение сервиса

Требование ТЗ	Где	Как проверить	Статус
Обучение операторов работе с ПО системы-112	эмулятор АРМ-112: frontend/src/emulator/, frontend/src/intake/	демо-сценарий PRD раздел 5	да
Отработка навыков обработки происшествий и ЧС	96 ситуаций из билетов, data/seed/tickets.json	pytest backend/tests/importers	да
Контроль качества через автоматизированную оценку	domain/evaluation, разбор попытки	pytest backend/tests/domain/evaluation	да
Моделирование ситуаций с эмуляцией вызовов	режимы «Реагирование на карточку» и «Приём вызова»	e2e dds.spec.ts, call_intake.spec.ts	да
Управление учебным процессом с уровнями сложности	сложность 1-3 в занятии, адаптивный подбор	pytest backend/tests/domain/analytics	да
Анализ эффективности обучения на основе машинного обучения	backend/app/analytics/, прогноз готовности	python -m app.domain.analytics.train_readiness	да
Реалистичная имитация интерфейса без доступа к внешним сетям	экраны повторяют АРМ-112 по скриншотам заказчика; внешних запросов нет	e2e-тест «нет внешних запросов»	да
Автоматизированный контроль временных нормативов и ручного ввода (сравнение с эталоном)	evaluation/timing.py, evaluation/text.py	тесты 30/45/60 с при нормативе 30	да
Снижение нагрузки на преподавателя за счёт	backend/app/scenarios/, генерация из билетов и по	pytest backend/tests/domain/scenarios	да

генерации сценариев и первичной оценки	фразе		
Защита данных пользователей и учебных материалов	роли, аудит, TLS, шифрование паролей	pytest backend/tests/api/test_admin.py	да
Масштабируемость под различные группы пользователей	группы, службы, профили	pytest backend/tests/api/test_sessions.py	да
Возможность удалённого контроля учебного процесса	живой мониторинг занятия по WebSocket	e2e teacher.spec.ts	да

3. Постановка задачи

Требование ТЗ	Где	Как проверить	Статус
Эмуляция рабочих процессов оператора 112 в локальном комплексе	режим «Приём вызова»	e2e call_intake.spec.ts	да
Имитация входящих вызовов (виртуальная IP-телефония или текстовые сообщения)	Asterisk + софтфон; текстовый путь при выключенной телефонии	bench_call.py, e2e	да
Инструменты управления обучением для преподавателя	кабинет преподавателя	e2e teacher.spec.ts	да
Система оценки действий на основе нейросетевого анализа	оценка: эталон, смысловая близость по эмбедингам, детекторы типичных ошибок	pytest backend/tests/domain/evaluation, tests/providers	да
Работа с различными уровнями сложности	сложность занятия, адаптивный подбор	tests/domain/analytics	да
Имитация функционала и интерфейса системы-112 в изолированном контуре	весь продукт работает без интернета	запуск без сети, e2e «нет внешних запросов»	да
Контроль выполнения рабочих функций (тайминг и регламент)	нормативы, статусы, детекторы нарушений из памятки	tests/domain/evaluation/test_detectors*	да
Оценка на основе анализа ручного ввода текста	сравнение описания и комментариев с эталоном, грамотность	tests/domain/evaluation	да
Формирование отчётности по результатам обучения	отчёт занятия, экспорт PDF, XLSX, CSV	pytest backend/tests/reports	да

4. Границы решения

Требование ТЗ	Где	Как проверить	Статус
Не обрабатывать реальные экстренные вызовы	учебный контур, вызовы только внутри стенда	docs/LIMITATIONS.md	да
Не содержать функционал основной системы-112	только имитация, без интеграции с ПОВ-112	архитектура	да
Не работать с конфиденциальной информацией вне учебного процесса	данные только учебные, ФИО скрыто до действия с аудитом	tests/api/test_admin.py	да
Не выполнять функции реального диспетчерского центра	нет связи с реальными службами	docs/LIMITATIONS.md	да

5. Общие требования к сервису

Требование ТЗ	Где	Как проверить	Статус
Эмуляция работы оператора системы-112 в реальном времени	оба режима занятий	демо-сценарий	да

Обработка вызовов через виртуальную IP-телефонию (VoIP)	Asterisk PJSIP, WebRTC-софтфон	живой звонок, bench_call.py	да
Система управления учебным процессом с распределением заданий	занятия, группы, назначения	e2e teacher.spec.ts	да
Контроль действий обучаемых (тайминг, регламент)	нормативы и машина статусов	tests/domain/evaluation/test_status_machine*	да
Оценка эффективности обучения на основе ИИ	комментарий ИИ, выжимка, прогноз	tests/providers, tests/domain/analytics	да
Ведение статистики по результатам обучения	прогресс обучающегося, аналитика группы	e2e analytics.spec.ts	да
Многоуровневая система аутентификации	пароль + ограничение попыток + роли; второго фактора нет	tests/test_auth.py	частично, TOTP в фазе E
Разграничение доступа к данным по ролям	зависимости ролей и проверки владельца	tests/api/test_admin.py, test_sessions.py	да
Защита каналов передачи данных	TLS на nginx, WSS, DTLS-SRTP в WebRTC	конфигурация deploy/nginx, deploy/asterisk	да
Резервное копирование не реже раза в сутки	сервис backup, крон 03:00, хранение копий	scripts/restore_test.sh	да
Интеграция с форматами данных и визуальными стандартами ГБУ «Система 112»	классификатор, статусы и экраны заказчика	docs/DATASET.md, сверка со скриншотами	да
Интеграция с локальной системой управления доступом	не реализовано , вход только внутренний	—	нет, задача #44
Интеграция с локальными средствами мониторинга	/metrics для Prometheus, экран состояния системы	curl /metrics, экран администратора	частично, дашборд Grafana в фазе E

6. Опциональные требования

Требование ТЗ	Где	Как проверить	Статус
Дополнительные модули аналитики (графики, тепловые карты ошибок)	аналитика группы, тепловая карта	e2e analytics.spec.ts	да
Экспорт отчётности в Excel и PDF	backend/app/reports/export.py	pytest backend/tests/reports	да
Механизм импорта обновлений учебных материалов пакетом	загрузка материалов в справочник есть, пакетного обновления сценариев нет	—	частично
Адаптивный веб-интерфейс для мобильных устройств	кабинеты адаптивны; эмулятор APM-112 рассчитан на десктоп и планшет	проверка на ширине 390 px	частично, по ТЗ допустимо
Внутренний API для расширения функционала	REST с OpenAPI, backend/app/openapi_export.py	/api/openapi.json	да

7. Требования к производительности

Требование ТЗ	Где	Как проверить	Статус
Отклик интерфейса не более 2 с при 100 пользователях	асинхронный бэкэнд, индексы, отчёты в фоне	load/load_http.py browse - -users 100, docs/PERFORMANCE.md: p95 1,8 с	да
Не менее 20 одновременных сессий	pub/sub, отдельные каналы занятия	load/load_http.py cards --users 20, docs/PERFORMANCE.md: p95 1,1	да

		с	
Задержка голоса не более 150 мс	локальная сеть, кодеки Opus и G.722	статистика WebRTC на стенде: 2 мс, потерь нет (docs/PERFORMANCE.md)	да
Работа при сбоях сети до 30 с с восстановлением сессий	черновики, идемпотентная отправка, догон событий по after_seq	e2e с отключением сети	да
Скорость записи в БД не менее 100 операций в секунду	пакетная запись событий занятия	scripts/bench_db_writes.py, docs/PERFORMANCE.md: 825 событий/с	да
Формирование аналитических отчётов не более 30 с	агрегаты SQL, фоновая задача при долгом отчёте	тест на seed-истории	да
Буферизация данных при временных сбоях	черновик карточки в браузере и на сервере	e2e офлайн	да
Вертикальное масштабирование	ресурсы сервера, настройки воркеров	docs/ARCHITECTURE.md	да
Горизонтальное масштабирование (узлы локального кластера)	бэкенд без состояния, телефония отдельным узлом	docs/ARCHITECTURE.md, стенд из двух узлов	да
Минимальные требования к АРМ обучающегося	браузер и гарнитура, всё тяжёлое на сервере	docs/INSTALL.md	да
Минимальные требования к серверу (6 ядер, 32 ГБ, NVMe 512 ГБ)	ядро работает на CPU, модели в квантованном виде	docs/PERFORMANCE.md	да

8. Пользовательские требования: интерфейс

Требование ТЗ	Где	Как проверить	Статус
Интуитивная понятность для всех категорий пользователей	экраны повторяют привычный АРМ-112, учебное вынесено отдельно	проверка на стенде, docs/BUGS.md	да
Единая стилистика оформления	общая дизайн-система, токены темы	визуальная проверка	да
Адаптивность под различные устройства	кабинеты до 390 px, эмулятор от 1280 px	проверка ширины	частично
Локализация на русском языке	весь интерфейс на русском	визуальная проверка	да

9. Роль: администратор

Требование ТЗ	Где	Как проверить	Статус
Запускать и останавливать сервисы системы	backend/app/admin/health.py, перезапуск сервиса	e2e admin.spec.ts	да
Контролировать состояние компонентов в реальном времени	экран состояния системы	e2e admin.spec.ts	да
Настраивать параметры производительности	настройки системы	экран настроек	частично, набор параметров ограничен
Выполнять обновление ПО пакетно	обновление образами, инструкция	docs/INSTALL.md	частично, из интерфейса не запускается
Проводить резервное копирование	backend/app/admin/backups.py, ручная копия	scripts/restore_test.sh	да
Создавать учётные записи всех категорий	администрирование пользователей	e2e admin.spec.ts	да
Назначать роли и права доступа	роль и служба у пользователя	tests/api/test_admin.py	да
Блокировать и разблокировать учётные записи	флаг блокировки	tests/api/test_admin.py	да
Настраивать параметры виртуальной IP-телефонии	настройки телефонии	экран настроек	да
Конфигурировать взаимодействие с локальной	настройки подключения в .env	docs/INSTALL.md	частично, из интерфейса не меняется

базой данных			
Управлять настройками безопасности и политиками доступа	политики паролей и ролей	экран настроек	частично
Настраивать параметры резервного копирования	расписание и хранение копий	экран настроек	да
Конфигурировать параметры журналирования	уровень логов	экран настроек	да
Просматривать системные журналы	журнал аудита и логи сервисов	e2e admin.spec.ts	да
Анализировать статистику использования системы	сводка активности	экран состояния	да
Формировать отчёты об ошибках и сбоях	оповещения администратора	экран состояния	частично, выгрузки отчёта нет
Отслеживать нагрузку на сервер	загрузка процессора и памяти	экран состояния	да
Контролировать показатели работоспособности	проверки работоспособности сервисов	/api/admin/health	да
Управлять политиками безопасности и правилами доступа	роли и ограничения	tests/api/test_admin.py	частично
Контролировать целостность системы	проверка цепочки аудита	POST /api/admin/audit/verify	да
Вести аудит безопасности	backend/app/audit.py, неизменяемый журнал	tests/test_audit.py	да
Ограничение: не вмешиваться в учебный процесс (не менять оценки и сценарии)	администратор получает 403	tests/api/test_admin.py	да
Ограничение: не удалять критичные данные без копии	удаление закрыто, есть резервные копии	tests/api/test_admin.py	да
Ограничение: не менять базовые настройки безопасности без прав	разграничение по роли	tests/api/test_admin.py	частично
Ограничение: доступ к персональным данным по необходимости	ФИО скрыто, показ пишется в аудит	tests/api/test_admin.py	да

10. Роль: преподаватель

Требование ТЗ	Где	Как проверить	Статус
Создавать, редактировать и утверждать сценарии, в том числе сгенерированные нейросетью	экран сценариев, генерация, версии, утверждение	e2e scenarios.spec.ts	да
Назначать задания и группы	группы и занятия	e2e teacher.spec.ts	да
Контролировать выполнение задач в реальном времени	живой мониторинг	e2e teacher.spec.ts	да
Отслеживать прогресс и историю успеваемости	аналитика группы, история занятий	e2e analytics.spec.ts	да
Создавать материалы и загружать ресурсы	загрузка методических материалов в справочник	tests/api/test_scenarios.py	да
Классифицировать задания по уровням сложности	сложность сценария и занятия	tests/domain/scenarios	да
Редактировать и удалять неактуальные сценарии	правка и архив сценария	e2e scenarios.spec.ts	да
Просматривать результаты (оценка ИИ и экспертная)	разбор попытки, изменение оценки с причиной	tests/api/test_overrides.py	да
Формировать отчёты по успеваемости и статистику	отчёт занятия и экспорт	pytest backend/tests/reports	да
Получать аналитические рекомендации (инсайты по типичным ошибкам группы)	топ ошибок и выжимка	e2e analytics.spec.ts	да
Мониторинг выполнения	плитки обучающихся	e2e teacher.spec.ts	да

заданий в реальном времени			
Предоставлять обратную связь через интерфейс	комментарий к попытке	tests/api/test_overrides.py	да
Давать рекомендации по улучшению навыков	рекомендации в прогрессе обучающегося	e2e analytics.spec.ts	да
Определять параметры сценариев (тип, локация, сложность)	форма генерации сценария	e2e scenarios.spec.ts	да
Устанавливать тайминг, по умолчанию 30 секунд	норматив занятия, значение по умолчанию 30	tests/api/test_sessions.py	да
Настраивать критерии успешности (пороги ошибок, требования к синтаксису)	порог зачёта, веса составляющих, проверка грамотности	tests/domain/evaluation	да
Ограничение: не менять системные настройки платформы	403 на админские методы	tests/api/test_sessions.py	да
Ограничение: не удалять критичные системные данные	удаление закрыто	tests/api/test_admin.py	да
Ограничение: не вмешиваться в работу других преподавателей	доступ только к своим группам и занятиям	tests/api/test_sessions.py	да
Ограничение: изменение оценок только с записью в журнал аудита	причина обязательна, запись в аудит	tests/api/test_overrides.py	да

11. Роль: обучающийся

Требование ТЗ	Где	Как проверить	Статус
Доступ к назначенным материалам и сценариям	мои задания	e2e dds.spec.ts	да
Выполнение практических заданий в режиме эмуляции	оба режима	e2e	да
Получение обратной связи по результатам	разбор попытки	e2e	да
Отслеживание прогресса и истории ошибок	мой прогресс	e2e analytics.spec.ts	да
Сохранность результатов в локальном профиле	история попыток в базе	tests/api	да
Доступ к списку назначенных сценариев	мои задания	e2e	да
Просмотр инструкций и методических материалов	справочник с поиском по памятке	tests/api/test_reference.py	да
Выполнение практических заданий (заполнение карточек)	карточка оператора 112	e2e call_intake.spec.ts	да
Сохранение промежуточных результатов	черновик карточки	e2e офлайн	да
Принимать входящие вызовы и сообщения в режиме обучения	софтфон и текстовый путь	e2e softphone.spec.ts	да
Использовать инструменты эмуляции АРМ-112	журнал и карточка	e2e dds.spec.ts	да
Вводить текстовые ответы в карточку согласно сценарию	комментарий к статусам, описание	tests/domain/evaluation	да
Применять стандартные процедуры обработки вызовов	статусы и регламент из памятки	tests/domain/evaluation	да
Просматривать свои результаты (оценки, время реакции)	разбор и прогресс	e2e	да
Отслеживать статистику выполнения заданий	мой прогресс	e2e	да
Видеть рекомендации системы	рекомендации в прогрессе	e2e	да

Анализировать свои ошибки на основе отчётов ИИ	разбор с объяснением и комментарием ИИ	e2e	да
Выбирать доступные учебные модули	список назначенных занятий	e2e	да
Контролировать время выполнения (таймер в интерфейсе)	таймер в карточке и в журнале	e2e	да
Ограничение: нет доступа к функциям преподавателя и администратора	403 и «Нет доступа»	tests/api	да
Ограничение: не менять учебные материалы	доступ только на чтение	tests/api/test_scenarios.py	да
Ограничение: не удалять результаты	удаление закрыто	tests/api	да
Ограничение: не менять настройки системы	403	tests/api/test_admin.py	да
Ограничение: не видеть результаты других обучающихся	проверка владельца попытки	tests/api	да
Защита персональных данных обучающегося	минимум данных, аудит доступа	tests/api/test_admin.py	да
Конфиденциальность результатов обучения	доступ только своим и преподавателю занятия	tests/api	да
Надёжное хранение истории в локальной базе	PostgreSQL, резервные копии	scripts/restore_test.sh	да
Аутентификация при каждом входе	вход по логину и паролю, сессия по токenu	tests/test_auth.py	да

12. Нефункциональные требования

Требование ТЗ	Где	Как проверить	Статус
Резервное копирование не реже раза в сутки	сервис backup	scripts/restore_test.sh	да
Отказоустойчивость при выходе из строя отдельных узлов	телефония и модели отдельными узлами, перезапуск сервисов	docs/ARCHITECTURE.md	частично, кластера нет
Автоматическое восстановление сервисов после сбоев	restart: unless-stopped, проверки работоспособности	остановка контейнера и наблюдение	да
Мониторинг критических параметров в реальном времени	экран состояния, /metrics	экран администратора	да
Журналирование всех значимых событий	аудит действий	tests/test_audit.py	да
Многоуровневая аутентификация в рамках локальной сети	пароль и роли, второй фактор не сделан	tests/test_auth.py	частично, TOTP в фазе E
Шифрование передаваемых данных (TLS внутри контура)	TLS на nginx, WSS	конфигурация	да
Разграничение доступа по ролям (RBAC)	роли и владение объектами	tests/api	да
Защита от несанкционированного доступа	ограничение попыток входа, заголовки безопасности	tests/test_auth.py	да
Аудит всех действий пользователей	неизменяемый журнал с цепочкой хэшей	POST /api/admin/audit/verify	да
Хранение журналов безопасности не менее 6 месяцев	записи не удаляются, ретенция описана	docs/ARCHITECTURE.md	да
ОС: Windows 10/11 и Ubuntu 20.04 и выше	Docker Compose, проверено на Windows и Ubuntu	docs/INSTALL.md	да
Браузеры: Chrome, Firefox, Яндекс.Браузер	проверка в трёх браузерах	e2e и ручная проверка	частично, Яндекс.Браузер проверен вручную
СУБД PostgreSQL 12 и выше	PostgreSQL 16 с pgvector	docker compose ps	да

Поддержка IP-телефонии через локальный SIP-сервер	Asterisk в контуре	живой звонок	да
Документированные процедуры установки, настройки и восстановления	docs/INSTALL.md	сторонний человек ставит по инструкции	да
Инструменты диагностики и оповещение администратора об ошибках	оповещения и экран состояния	экран администратора	да

13. Сценарии из ТЗ и ответы заказчика

Требование	Где	Как проверить	Статус
Сценарий «Настройка учебной среды»: выбор категорий, генерация эталонов, подтверждение преподавателем, ролевая модель ленты, предпросмотр, коррекция комментарием, обучение на материалах, проверка грамматики	экран сценариев и форма занятия	e2e scenarios.spec.ts	да
Сценарий «Практическое занятие на АРМ-112 (карточки)»: вход, выбор категорий, старт, имитация звонка, заполнение карточки, завершение, отчёт	режим «Приём вызова»	e2e call_intake.spec.ts	да
Сценарий «Действия с карточками»: выбор источника карточек (сгенерированные, созданные обучающимися, смешанные), случайная выдача, действия с карточкой и ввод текста, отчёт	режим «Реагирование на карточку»	e2e dds.spec.ts	да
Ответ заказчика: карточка приходит диспетчеру уже заполненной	режим ДДС	e2e dds.spec.ts	да
Ответ заказчика: диспетчер проверяет данные карточки на ошибки оператора 112	заложенные ошибки в карточке, отметка полей, составляющая «Проверка данных»	test_data_check.py, test_distort.py, e2e dds.spec.ts	да
Ответ заказчика: диспетчер совершает исходящие звонки руководителям служб	звонок дежурному службы из карточки, составляющая «Звонки в службы»	test_service_call.py, tests/telephony/test_service_calls.py, e2e dds.spec.ts	да
Инструкция заказчика: статусы телефонии оператора и постобработка вызова	не реализовано	—	нет, задача #38
Ответ заказчика 23.09: диспетчер звонит заявителю напрямую по номеру из карточки	обратный звонок из карточки, без называния номера карточки	tests/telephony/test_service_calls.py, e2e dds.spec.ts	да
Ответ заказчика 21.09: старший группы докладывает диспетчеру по телефону о ходе работ	входящие доклады бригады, ответ дежурного, оценка по докладам	test_service_call.py, e2e dds.spec.ts	да
Ответ заказчика 23.09: карточки приходят диспетчеру потоком, норматив идёт у каждой	выдача по числу карточек уровня сложности, норматив с момента «Добавлена»	tests/api/test_attempts.py, e2e dds.spec.ts	да

Ответ заказчика 22.09: в карточке ДДС службы не добавляются, диспетчер видит остальные и может связаться	панель служб только для чтения, звонок дежурному	e2e.dds.spec.ts	да
Требование заказчика 21.09: карточка и реестр повторяют живой АРМ-112 по составу и цветам	эмулятор АРМ: колонки реестра, строка статуса, полоса служб	сверка со скриншотами заказчика от 17.09.2026	да

14. Требования к решению и сдаче

Требование ТЗ	Где	Как проверить	Статус
Открытый исходный код без обфускации	репозиторий проекта	просмотр	да
Перечень использованных библиотек и компонентов	docs/LIBRARIES.md	просмотр	да
Комментарии к сложным техническим решениям	комментарии в коде, docs/DECISIONS.md	просмотр	да
Описание методов обработки данных	docs/METHODS.md	просмотр	да
Условия и ограничения решения	docs/LIMITATIONS.md	просмотр	да
Инструкции по сборке, установке и запуску	docs/INSTALL.md, README.md	установка с нуля	да
Описание функциональной и компонентной архитектуры	docs/ARCHITECTURE.md	просмотр	да
Соответствие 149-ФЗ и 152-ФЗ	минимум персональных данных, аудит, разграничение доступа	docs/LIMITATIONS.md	да
Презентация в формате pptx или pdf	шаблон организаторов	сдача	частично, готовится
Ссылка на прототип для проверки	стенд с демо-доступом	открыть ссылку	да
Сопроводительная документация в docx или pdf	docs/submission/ (11 разделов, 53 страницы), сборка scripts/build_docs.py	просмотр	да

Что закрывает оставшиеся пункты

Пункт	Задача
Статусы телефонии и постобработка вызова	#38
Замеры производительности (100 пользователей, 20 сессий, запись в БД, голос)	#12, волна 11
Сборка документации в DOCX и PDF	#51
Второй фактор аутентификации (TOTP)	фаза Е
Интеграция с локальной системой управления доступом (LDAP)	#44
Пакетное обновление сценариев	фаза Е